In [1]:
```
# ============================================================
# MEMBER 3
# EDA & STATISTICAL ANALYSIS
# MEMORY-EFFICIENT VERSION
# STEPS 22–56
# ============================================================

import pandas as pd
import numpy as np
import os
import sys
import gc

from IPython.display import display

# ============================================================
# SETTINGS
# ============================================================

INPUT_FILE = r"C:\Users\admin\OneDrive\Desktop\DS- Activity 1\backblaze_analytic

OUTPUT_FOLDER = "output/eda_results"

CHUNK_SIZE = 100_000

# Sample size used for operations that require
# storing observations such as quantiles/correlation.
SAMPLE_SIZE = 500_000

os.makedirs(
    OUTPUT_FOLDER,
    exist_ok=True
)

pd.set_option(
    "display.max_columns",
    None
)

pd.set_option(
    "display.max_rows",
    100
)

pd.set_option(
    "display.width",
    200
)


# ============================================================
# HELPER — JUPYTER + TERMINAL + CSV
# ============================================================

def show_and_save(
    data,
    filename,
    title
```

```
):

    print("\n" + "=" * 70)
    print(title)
    print("=" * 70)

    display(data)

    try:

        sys.__stdout__.write(
            "\n" +
            "=" * 70 +
            "\n" +
            title +
            "\n" +
            "=" * 70 +
            "\n" +
            data.to_string(
                index=False
            ) +
            "\n"
        )

        sys.__stdout__.flush()

    except Exception:
        pass

    filepath = os.path.join(
        OUTPUT_FOLDER,
        filename
    )

    data.to_csv(
        filepath,
        index=False
    )

    print(
        "\nSaved CSV:"
    )

    print(
        filepath
    )

    try:

        sys.__stdout__.write(
            f"\nSaved CSV: {filepath}\n"
        )

        sys.__stdout__.flush()

    except Exception:
        pass

    return data
```

In [2]:
```
# ============================================================
# STEP 22 — LOAD DATASET INFORMATION
# ============================================================

print("\n" + "=" * 70)
print("STEP 22 — LOAD CLEANED DATASET")
print("=" * 70)

# Read only a small sample
sample_df = pd.read_csv(
    INPUT_FILE,
    nrows=1000
)

DATA_COLUMNS = (
    sample_df.columns.tolist()
)

NUMERICAL_COLUMNS = (
    sample_df
    .select_dtypes(
        include=np.number
    )
    .columns
    .tolist()
)

CATEGORICAL_COLUMNS = (
    sample_df
    .select_dtypes(
        exclude=np.number
    )
    .columns
    .tolist()
)

# Count rows without loading everything
total_rows = 0

for chunk in pd.read_csv(
    INPUT_FILE,
    chunksize=CHUNK_SIZE,
    usecols=[DATA_COLUMNS[0]]
):

    total_rows += len(chunk)

dataset_information = pd.DataFrame({

    "Metric": [

        "Total Rows",

        "Total Columns",

        "Numerical Columns",

        "Categorical Columns",
```

```
        "Chunk Size"
    ],

    "Value": [

        total_rows,

        len(DATA_COLUMNS),

        len(NUMERICAL_COLUMNS),

        len(CATEGORICAL_COLUMNS),

        CHUNK_SIZE
    ]
})

show_and_save(
    dataset_information,
    "step_22_dataset_information.csv",
    "STEP 22 — DATASET INFORMATION"
)
```

```
======================================================================
STEP 22 — LOAD CLEANED DATASET
======================================================================

======================================================================
STEP 22 — DATASET INFORMATION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Metric</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

Metric	Value
Total Rows	30943146
Total Columns	19
Numerical Columns	16
Categorical Columns	3
Chunk Size	100000

```
Saved CSV:
output/eda_results\step_22_dataset_information.csv
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Metric</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

Metric	Value
Total Rows	30943146
Total Columns	19
Numerical Columns	16
Categorical Columns	3
Chunk Size	100000

In [3]:
```
# ============================================================
# STEP 23 — COLUMN INFORMATION
# ============================================================

column_information = pd.DataFrame({

    "Column_Number":
        range(
            1,
            len(DATA_COLUMNS) + 1
        ),

    "Column_Name":
        DATA_COLUMNS,

    "Data_Type":
        sample_df.dtypes
        .astype(str)
        .values
})

show_and_save(
    column_information,
    "step_23_column_information.csv",
    "STEP 23 — COLUMN INFORMATION"
)
```

```
======================================================================
STEP 23 — COLUMN INFORMATION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Column_Number</th> <th>Column_Name</th> <th>Data_Type</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

1	date	str
2	serial_number	str
3	model	str
4	capacity_bytes	int64
5	failure	int64
6	smart_5_normalized	float64
7	smart_5_raw	float64
8	smart_9_normalized	float64
9	smart_9_raw	float64
10	smart_90_normalized	float64
11	smart_90_raw	float64
12	smart_187_normalized	float64
13	smart_187_raw	float64
14	smart_188_normalized	float64
15	smart_188_raw	float64
16	smart_197_normalized	float64
17	smart_197_raw	float64
18	smart_198_normalized	float64
19	smart_198_raw	float64

```
Saved CSV:
output/eda_results\step_23_column_information.csv
```

Out[3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Column_Number</th> <th>Column_Name</th> <th>Data_Type</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

1	date	str
2	serial_number	str
3	model	str
4	capacity_bytes	int64
5	failure	int64
6	smart_5_normalized	float64
7	smart_5_raw	float64
8	smart_9_normalized	float64
9	smart_9_raw	float64
10	smart_90_normalized	float64
11	smart_90_raw	float64
12	smart_187_normalized	float64
13	smart_187_raw	float64
14	smart_188_normalized	float64
15	smart_188_raw	float64
16	smart_197_normalized	float64
17	smart_197_raw	float64
18	smart_198_normalized	float64
19	smart_198_raw	float64

In [4]:
```
# ============================================================
# STEP 24 — MISSING VALUE ANALYSIS
# ============================================================

missing_count = pd.Series(
    0,
    index=DATA_COLUMNS,
    dtype="int64"
)

for chunk in pd.read_csv(
    INPUT_FILE,
    chunksize=CHUNK_SIZE
):
```

```
    missing_count = (
        missing_count
        +
        chunk.isnull().sum()
    )

missing_percentage = (
    missing_count
    /
    total_rows
    *
    100
)

missing_result = pd.DataFrame({

    "Column":
        DATA_COLUMNS,

    "Missing_Count":
        missing_count.values,

    "Missing_Percentage":
        missing_percentage.values
})

show_and_save(
    missing_result,
    "step_24_missing_value_analysis.csv",
    "STEP 24 — MISSING VALUE ANALYSIS"
)
```

```
======================================================================
STEP 24 — MISSING VALUE ANALYSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Column</th> <th>Missing_Count</th> <th>Missing_Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	0	0.000000
serial_number	0	0.000000
model	0	0.000000
capacity_bytes	0	0.000000
failure	0	0.000000
smart_5_normalized	208717	0.674518
smart_5_raw	208717	0.674518
smart_9_normalized	39856	0.128804
smart_9_raw	39856	0.128804
smart_90_normalized	26718632	86.347497
smart_90_raw	26718632	86.347497
smart_187_normalized	20675970	66.819224
smart_187_raw	20675970	66.819224
smart_188_normalized	20703124	66.906978
smart_188_raw	20703124	66.906978
smart_197_normalized	775206	2.505259
smart_197_raw	775206	2.505259
smart_198_normalized	237727	0.768270
smart_198_raw	237727	0.768270

```
Saved CSV:
output/eda_results\step_24_missing_value_analysis.csv
```

Out[4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Column</th> <th>Missing_Count</th> <th>Missing_Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	0	0.000000
serial_number	0	0.000000
model	0	0.000000
capacity_bytes	0	0.000000
failure	0	0.000000
smart_5_normalized	208717	0.674518
smart_5_raw	208717	0.674518
smart_9_normalized	39856	0.128804
smart_9_raw	39856	0.128804
smart_90_normalized	26718632	86.347497
smart_90_raw	26718632	86.347497
smart_187_normalized	20675970	66.819224
smart_187_raw	20675970	66.819224
smart_188_normalized	20703124	66.906978
smart_188_raw	20703124	66.906978
smart_197_normalized	775206	2.505259
smart_197_raw	775206	2.505259
smart_198_normalized	237727	0.768270
smart_198_raw	237727	0.768270

In [5]:
```
# ============================================================
# STEP 25 — DUPLICATE ANALYSIS
# MEMORY-EFFICIENT
# ============================================================

print("\n" + "=" * 70)
print("STEP 25 — DUPLICATE ANALYSIS")
print("=" * 70)

seen_hashes = set()

duplicate_count = 0

processed_rows = 0
```

```
chunk_number = 0

for chunk in pd.read_csv(
    INPUT_FILE,
    chunksize=CHUNK_SIZE
):

    chunk_number += 1

    row_hashes = (
        pd.util.hash_pandas_object(
            chunk,
            index=False
        )
    )

    for row_hash in row_hashes:

        row_hash = int(row_hash)

        if row_hash in seen_hashes:

            duplicate_count += 1

        else:

            seen_hashes.add(
                row_hash
            )

    processed_rows += len(chunk)

    message = (

        f"Chunk {chunk_number}: "
        f"{processed_rows:,} rows | "
        f"Duplicates: "
        f"{duplicate_count:,}"
    )

    print(message)

    try:

        sys.__stdout__.write(
            message + "\n"
        )

        sys.__stdout__.flush()

    except Exception:
        pass

duplicate_percentage = (

    duplicate_count
    /
    total_rows
    *
    100
```

```
)

duplicate_result = pd.DataFrame({

    "Metric": [

        "Total Rows",

        "Duplicate Rows",

        "Duplicate Percentage"
    ],

    "Value": [

        total_rows,

        duplicate_count,

        duplicate_percentage
    ]
})

show_and_save(
    duplicate_result,
    "step_25_duplicate_analysis.csv",
    "STEP 25 — DUPLICATE ANALYSIS"
)

del seen_hashes

gc.collect()
```

```
======================================================================
STEP 25 — DUPLICATE ANALYSIS
======================================================================
Chunk 1: 100,000 rows | Duplicates: 0
Chunk 2: 200,000 rows | Duplicates: 0
Chunk 3: 300,000 rows | Duplicates: 0
Chunk 4: 400,000 rows | Duplicates: 0
Chunk 5: 500,000 rows | Duplicates: 0
Chunk 6: 600,000 rows | Duplicates: 0
Chunk 7: 700,000 rows | Duplicates: 0
Chunk 8: 800,000 rows | Duplicates: 0
Chunk 9: 900,000 rows | Duplicates: 0
Chunk 10: 1,000,000 rows | Duplicates: 0
Chunk 11: 1,100,000 rows | Duplicates: 0
Chunk 12: 1,200,000 rows | Duplicates: 0
Chunk 13: 1,300,000 rows | Duplicates: 0
Chunk 14: 1,400,000 rows | Duplicates: 0
Chunk 15: 1,500,000 rows | Duplicates: 0
Chunk 16: 1,600,000 rows | Duplicates: 0
Chunk 17: 1,700,000 rows | Duplicates: 0
Chunk 18: 1,800,000 rows | Duplicates: 0
Chunk 19: 1,900,000 rows | Duplicates: 0
Chunk 20: 2,000,000 rows | Duplicates: 0
Chunk 21: 2,100,000 rows | Duplicates: 0
Chunk 22: 2,200,000 rows | Duplicates: 0
Chunk 23: 2,300,000 rows | Duplicates: 0
Chunk 24: 2,400,000 rows | Duplicates: 0
Chunk 25: 2,500,000 rows | Duplicates: 0
Chunk 26: 2,600,000 rows | Duplicates: 0
Chunk 27: 2,700,000 rows | Duplicates: 0
Chunk 28: 2,800,000 rows | Duplicates: 0
Chunk 29: 2,900,000 rows | Duplicates: 0
Chunk 30: 3,000,000 rows | Duplicates: 0
Chunk 31: 3,100,000 rows | Duplicates: 0
Chunk 32: 3,200,000 rows | Duplicates: 0
Chunk 33: 3,300,000 rows | Duplicates: 0
Chunk 34: 3,400,000 rows | Duplicates: 0
Chunk 35: 3,500,000 rows | Duplicates: 0
Chunk 36: 3,600,000 rows | Duplicates: 0
Chunk 37: 3,700,000 rows | Duplicates: 0
Chunk 38: 3,800,000 rows | Duplicates: 0
Chunk 39: 3,900,000 rows | Duplicates: 0
Chunk 40: 4,000,000 rows | Duplicates: 0
Chunk 41: 4,100,000 rows | Duplicates: 0
Chunk 42: 4,200,000 rows | Duplicates: 0
Chunk 43: 4,300,000 rows | Duplicates: 0
Chunk 44: 4,400,000 rows | Duplicates: 0
Chunk 45: 4,500,000 rows | Duplicates: 0
Chunk 46: 4,600,000 rows | Duplicates: 0
Chunk 47: 4,700,000 rows | Duplicates: 0
Chunk 48: 4,800,000 rows | Duplicates: 0
Chunk 49: 4,900,000 rows | Duplicates: 0
Chunk 50: 5,000,000 rows | Duplicates: 0
Chunk 51: 5,100,000 rows | Duplicates: 0
Chunk 52: 5,200,000 rows | Duplicates: 0
Chunk 53: 5,300,000 rows | Duplicates: 0
Chunk 54: 5,400,000 rows | Duplicates: 0
Chunk 55: 5,500,000 rows | Duplicates: 0
Chunk 56: 5,600,000 rows | Duplicates: 0
Chunk 57: 5,700,000 rows | Duplicates: 0
```

```
Chunk 58: 5,800,000 rows | Duplicates: 0
Chunk 59: 5,900,000 rows | Duplicates: 0
Chunk 60: 6,000,000 rows | Duplicates: 0
Chunk 61: 6,100,000 rows | Duplicates: 0
Chunk 62: 6,200,000 rows | Duplicates: 0
Chunk 63: 6,300,000 rows | Duplicates: 0
Chunk 64: 6,400,000 rows | Duplicates: 0
Chunk 65: 6,500,000 rows | Duplicates: 0
Chunk 66: 6,600,000 rows | Duplicates: 0
Chunk 67: 6,700,000 rows | Duplicates: 0
Chunk 68: 6,800,000 rows | Duplicates: 0
Chunk 69: 6,900,000 rows | Duplicates: 0
Chunk 70: 7,000,000 rows | Duplicates: 0
Chunk 71: 7,100,000 rows | Duplicates: 0
Chunk 72: 7,200,000 rows | Duplicates: 0
Chunk 73: 7,300,000 rows | Duplicates: 0
Chunk 74: 7,400,000 rows | Duplicates: 0
Chunk 75: 7,500,000 rows | Duplicates: 0
Chunk 76: 7,600,000 rows | Duplicates: 0
Chunk 77: 7,700,000 rows | Duplicates: 0
Chunk 78: 7,800,000 rows | Duplicates: 0
Chunk 79: 7,900,000 rows | Duplicates: 0
Chunk 80: 8,000,000 rows | Duplicates: 0
Chunk 81: 8,100,000 rows | Duplicates: 0
Chunk 82: 8,200,000 rows | Duplicates: 0
Chunk 83: 8,300,000 rows | Duplicates: 0
Chunk 84: 8,400,000 rows | Duplicates: 0
Chunk 85: 8,500,000 rows | Duplicates: 0
Chunk 86: 8,600,000 rows | Duplicates: 0
Chunk 87: 8,700,000 rows | Duplicates: 0
Chunk 88: 8,800,000 rows | Duplicates: 0
Chunk 89: 8,900,000 rows | Duplicates: 0
Chunk 90: 9,000,000 rows | Duplicates: 0
Chunk 91: 9,100,000 rows | Duplicates: 0
Chunk 92: 9,200,000 rows | Duplicates: 0
Chunk 93: 9,300,000 rows | Duplicates: 0
Chunk 94: 9,400,000 rows | Duplicates: 0
Chunk 95: 9,500,000 rows | Duplicates: 0
Chunk 96: 9,600,000 rows | Duplicates: 0
Chunk 97: 9,700,000 rows | Duplicates: 0
Chunk 98: 9,800,000 rows | Duplicates: 0
Chunk 99: 9,900,000 rows | Duplicates: 0
Chunk 100: 10,000,000 rows | Duplicates: 0
Chunk 101: 10,100,000 rows | Duplicates: 0
Chunk 102: 10,200,000 rows | Duplicates: 0
Chunk 103: 10,300,000 rows | Duplicates: 0
Chunk 104: 10,400,000 rows | Duplicates: 0
Chunk 105: 10,500,000 rows | Duplicates: 0
Chunk 106: 10,600,000 rows | Duplicates: 0
Chunk 107: 10,700,000 rows | Duplicates: 0
Chunk 108: 10,800,000 rows | Duplicates: 0
Chunk 109: 10,900,000 rows | Duplicates: 0
Chunk 110: 11,000,000 rows | Duplicates: 0
Chunk 111: 11,100,000 rows | Duplicates: 0
Chunk 112: 11,200,000 rows | Duplicates: 0
Chunk 113: 11,300,000 rows | Duplicates: 0
Chunk 114: 11,400,000 rows | Duplicates: 0
Chunk 115: 11,500,000 rows | Duplicates: 0
Chunk 116: 11,600,000 rows | Duplicates: 0
Chunk 117: 11,700,000 rows | Duplicates: 0
```

```
Chunk 118: 11,800,000 rows | Duplicates: 0
Chunk 119: 11,900,000 rows | Duplicates: 0
Chunk 120: 12,000,000 rows | Duplicates: 0
Chunk 121: 12,100,000 rows | Duplicates: 0
Chunk 122: 12,200,000 rows | Duplicates: 0
Chunk 123: 12,300,000 rows | Duplicates: 0
Chunk 124: 12,400,000 rows | Duplicates: 0
Chunk 125: 12,500,000 rows | Duplicates: 0
Chunk 126: 12,600,000 rows | Duplicates: 0
Chunk 127: 12,700,000 rows | Duplicates: 0
Chunk 128: 12,800,000 rows | Duplicates: 0
Chunk 129: 12,900,000 rows | Duplicates: 0
Chunk 130: 13,000,000 rows | Duplicates: 0
Chunk 131: 13,100,000 rows | Duplicates: 0
Chunk 132: 13,200,000 rows | Duplicates: 0
Chunk 133: 13,300,000 rows | Duplicates: 0
Chunk 134: 13,400,000 rows | Duplicates: 0
Chunk 135: 13,500,000 rows | Duplicates: 0
Chunk 136: 13,600,000 rows | Duplicates: 0
Chunk 137: 13,700,000 rows | Duplicates: 0
Chunk 138: 13,800,000 rows | Duplicates: 0
Chunk 139: 13,900,000 rows | Duplicates: 0
Chunk 140: 14,000,000 rows | Duplicates: 0
Chunk 141: 14,100,000 rows | Duplicates: 0
Chunk 142: 14,200,000 rows | Duplicates: 0
Chunk 143: 14,300,000 rows | Duplicates: 0
Chunk 144: 14,400,000 rows | Duplicates: 0
Chunk 145: 14,500,000 rows | Duplicates: 0
Chunk 146: 14,600,000 rows | Duplicates: 0
Chunk 147: 14,700,000 rows | Duplicates: 0
Chunk 148: 14,800,000 rows | Duplicates: 0
Chunk 149: 14,900,000 rows | Duplicates: 0
Chunk 150: 15,000,000 rows | Duplicates: 0
Chunk 151: 15,100,000 rows | Duplicates: 0
Chunk 152: 15,200,000 rows | Duplicates: 0
Chunk 153: 15,300,000 rows | Duplicates: 0
Chunk 154: 15,400,000 rows | Duplicates: 0
Chunk 155: 15,500,000 rows | Duplicates: 0
Chunk 156: 15,600,000 rows | Duplicates: 0
Chunk 157: 15,700,000 rows | Duplicates: 0
Chunk 158: 15,800,000 rows | Duplicates: 0
Chunk 159: 15,900,000 rows | Duplicates: 0
Chunk 160: 16,000,000 rows | Duplicates: 0
Chunk 161: 16,100,000 rows | Duplicates: 0
Chunk 162: 16,200,000 rows | Duplicates: 0
Chunk 163: 16,300,000 rows | Duplicates: 0
Chunk 164: 16,400,000 rows | Duplicates: 0
Chunk 165: 16,500,000 rows | Duplicates: 0
Chunk 166: 16,600,000 rows | Duplicates: 0
Chunk 167: 16,700,000 rows | Duplicates: 0
Chunk 168: 16,800,000 rows | Duplicates: 0
Chunk 169: 16,900,000 rows | Duplicates: 0
Chunk 170: 17,000,000 rows | Duplicates: 0
Chunk 171: 17,100,000 rows | Duplicates: 0
Chunk 172: 17,200,000 rows | Duplicates: 0
Chunk 173: 17,300,000 rows | Duplicates: 0
Chunk 174: 17,400,000 rows | Duplicates: 0
Chunk 175: 17,500,000 rows | Duplicates: 0
Chunk 176: 17,600,000 rows | Duplicates: 0
Chunk 177: 17,700,000 rows | Duplicates: 0
```

```
Chunk 178: 17,800,000 rows | Duplicates: 0
Chunk 179: 17,900,000 rows | Duplicates: 0
Chunk 180: 18,000,000 rows | Duplicates: 0
Chunk 181: 18,100,000 rows | Duplicates: 0
Chunk 182: 18,200,000 rows | Duplicates: 0
Chunk 183: 18,300,000 rows | Duplicates: 0
Chunk 184: 18,400,000 rows | Duplicates: 0
Chunk 185: 18,500,000 rows | Duplicates: 0
Chunk 186: 18,600,000 rows | Duplicates: 0
Chunk 187: 18,700,000 rows | Duplicates: 0
Chunk 188: 18,800,000 rows | Duplicates: 0
Chunk 189: 18,900,000 rows | Duplicates: 0
Chunk 190: 19,000,000 rows | Duplicates: 0
Chunk 191: 19,100,000 rows | Duplicates: 0
Chunk 192: 19,200,000 rows | Duplicates: 0
Chunk 193: 19,300,000 rows | Duplicates: 0
Chunk 194: 19,400,000 rows | Duplicates: 0
Chunk 195: 19,500,000 rows | Duplicates: 0
Chunk 196: 19,600,000 rows | Duplicates: 0
Chunk 197: 19,700,000 rows | Duplicates: 0
Chunk 198: 19,800,000 rows | Duplicates: 0
Chunk 199: 19,900,000 rows | Duplicates: 0
Chunk 200: 20,000,000 rows | Duplicates: 0
Chunk 201: 20,100,000 rows | Duplicates: 0
Chunk 202: 20,200,000 rows | Duplicates: 0
Chunk 203: 20,300,000 rows | Duplicates: 0
Chunk 204: 20,400,000 rows | Duplicates: 0
Chunk 205: 20,500,000 rows | Duplicates: 0
Chunk 206: 20,600,000 rows | Duplicates: 0
Chunk 207: 20,700,000 rows | Duplicates: 0
Chunk 208: 20,800,000 rows | Duplicates: 0
Chunk 209: 20,900,000 rows | Duplicates: 0
Chunk 210: 21,000,000 rows | Duplicates: 0
Chunk 211: 21,100,000 rows | Duplicates: 0
Chunk 212: 21,200,000 rows | Duplicates: 0
Chunk 213: 21,300,000 rows | Duplicates: 0
Chunk 214: 21,400,000 rows | Duplicates: 0
Chunk 215: 21,500,000 rows | Duplicates: 0
Chunk 216: 21,600,000 rows | Duplicates: 0
Chunk 217: 21,700,000 rows | Duplicates: 0
Chunk 218: 21,800,000 rows | Duplicates: 0
Chunk 219: 21,900,000 rows | Duplicates: 0
Chunk 220: 22,000,000 rows | Duplicates: 0
Chunk 221: 22,100,000 rows | Duplicates: 0
Chunk 222: 22,200,000 rows | Duplicates: 0
Chunk 223: 22,300,000 rows | Duplicates: 0
Chunk 224: 22,400,000 rows | Duplicates: 0
Chunk 225: 22,500,000 rows | Duplicates: 0
Chunk 226: 22,600,000 rows | Duplicates: 0
Chunk 227: 22,700,000 rows | Duplicates: 0
Chunk 228: 22,800,000 rows | Duplicates: 0
Chunk 229: 22,900,000 rows | Duplicates: 0
Chunk 230: 23,000,000 rows | Duplicates: 0
Chunk 231: 23,100,000 rows | Duplicates: 0
Chunk 232: 23,200,000 rows | Duplicates: 0
Chunk 233: 23,300,000 rows | Duplicates: 0
Chunk 234: 23,400,000 rows | Duplicates: 0
Chunk 235: 23,500,000 rows | Duplicates: 0
Chunk 236: 23,600,000 rows | Duplicates: 0
Chunk 237: 23,700,000 rows | Duplicates: 0
```

```
Chunk 238: 23,800,000 rows | Duplicates: 0
Chunk 239: 23,900,000 rows | Duplicates: 0
Chunk 240: 24,000,000 rows | Duplicates: 0
Chunk 241: 24,100,000 rows | Duplicates: 0
Chunk 242: 24,200,000 rows | Duplicates: 0
Chunk 243: 24,300,000 rows | Duplicates: 0
Chunk 244: 24,400,000 rows | Duplicates: 0
Chunk 245: 24,500,000 rows | Duplicates: 0
Chunk 246: 24,600,000 rows | Duplicates: 0
Chunk 247: 24,700,000 rows | Duplicates: 0
Chunk 248: 24,800,000 rows | Duplicates: 0
Chunk 249: 24,900,000 rows | Duplicates: 0
Chunk 250: 25,000,000 rows | Duplicates: 0
Chunk 251: 25,100,000 rows | Duplicates: 0
Chunk 252: 25,200,000 rows | Duplicates: 0
Chunk 253: 25,300,000 rows | Duplicates: 0
Chunk 254: 25,400,000 rows | Duplicates: 0
Chunk 255: 25,500,000 rows | Duplicates: 0
Chunk 256: 25,600,000 rows | Duplicates: 0
Chunk 257: 25,700,000 rows | Duplicates: 0
Chunk 258: 25,800,000 rows | Duplicates: 0
Chunk 259: 25,900,000 rows | Duplicates: 0
Chunk 260: 26,000,000 rows | Duplicates: 0
Chunk 261: 26,100,000 rows | Duplicates: 0
Chunk 262: 26,200,000 rows | Duplicates: 0
Chunk 263: 26,300,000 rows | Duplicates: 0
Chunk 264: 26,400,000 rows | Duplicates: 0
Chunk 265: 26,500,000 rows | Duplicates: 0
Chunk 266: 26,600,000 rows | Duplicates: 0
Chunk 267: 26,700,000 rows | Duplicates: 0
Chunk 268: 26,800,000 rows | Duplicates: 0
Chunk 269: 26,900,000 rows | Duplicates: 0
Chunk 270: 27,000,000 rows | Duplicates: 0
Chunk 271: 27,100,000 rows | Duplicates: 0
Chunk 272: 27,200,000 rows | Duplicates: 0
Chunk 273: 27,300,000 rows | Duplicates: 0
Chunk 274: 27,400,000 rows | Duplicates: 0
Chunk 275: 27,500,000 rows | Duplicates: 0
Chunk 276: 27,600,000 rows | Duplicates: 0
Chunk 277: 27,700,000 rows | Duplicates: 0
Chunk 278: 27,800,000 rows | Duplicates: 0
Chunk 279: 27,900,000 rows | Duplicates: 0
Chunk 280: 28,000,000 rows | Duplicates: 0
Chunk 281: 28,100,000 rows | Duplicates: 0
Chunk 282: 28,200,000 rows | Duplicates: 0
Chunk 283: 28,300,000 rows | Duplicates: 0
Chunk 284: 28,400,000 rows | Duplicates: 0
Chunk 285: 28,500,000 rows | Duplicates: 0
Chunk 286: 28,600,000 rows | Duplicates: 0
Chunk 287: 28,700,000 rows | Duplicates: 0
Chunk 288: 28,800,000 rows | Duplicates: 0
Chunk 289: 28,900,000 rows | Duplicates: 0
Chunk 290: 29,000,000 rows | Duplicates: 0
Chunk 291: 29,100,000 rows | Duplicates: 0
Chunk 292: 29,200,000 rows | Duplicates: 0
Chunk 293: 29,300,000 rows | Duplicates: 0
Chunk 294: 29,400,000 rows | Duplicates: 0
Chunk 295: 29,500,000 rows | Duplicates: 0
Chunk 296: 29,600,000 rows | Duplicates: 0
Chunk 297: 29,700,000 rows | Duplicates: 0
```

```
Chunk 298: 29,800,000 rows | Duplicates: 0
Chunk 299: 29,900,000 rows | Duplicates: 0
Chunk 300: 30,000,000 rows | Duplicates: 0
Chunk 301: 30,100,000 rows | Duplicates: 0
Chunk 302: 30,200,000 rows | Duplicates: 0
Chunk 303: 30,300,000 rows | Duplicates: 0
Chunk 304: 30,400,000 rows | Duplicates: 0
Chunk 305: 30,500,000 rows | Duplicates: 0
Chunk 306: 30,600,000 rows | Duplicates: 2,516
Chunk 307: 30,700,000 rows | Duplicates: 102,516
Chunk 308: 30,800,000 rows | Duplicates: 202,516
Chunk 309: 30,900,000 rows | Duplicates: 302,516
Chunk 310: 30,943,146 rows | Duplicates: 345,662

======================================================================
STEP 25 — DUPLICATE ANALYSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Metric</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Metric	Value
Total Rows	3.094315e+07
Duplicate Rows	3.456620e+05
Duplicate Percentage	1.117087e+00

```
Saved CSV:
output/eda_results\step_25_duplicate_analysis.csv
```

Out[5]: 169

In [6]:
```
# ============================================================
# STEP 26 — NUMERICAL FEATURES
# ============================================================

numeric_result = pd.DataFrame({

    "Numerical_Feature":
        NUMERICAL_COLUMNS
})

show_and_save(
    numeric_result,
    "step_26_numerical_features.csv",
    "STEP 26 — NUMERICAL FEATURES"
)
```

```
======================================================================
STEP 26 — NUMERICAL FEATURES
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Numerical_Feature</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	
failure	
smart_5_normalized	
smart_5_raw	
smart_9_normalized	
smart_9_raw	
smart_90_normalized	
smart_90_raw	
smart_187_normalized	
smart_187_raw	
smart_188_normalized	
smart_188_raw	
smart_197_normalized	
smart_197_raw	
smart_198_normalized	
smart_198_raw	

```
Saved CSV:
output/eda_results\step_26_numerical_features.csv
```

Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Numerical_Feature</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes
failure
smart_5_normalized
smart_5_raw
smart_9_normalized
smart_9_raw
smart_90_normalized
smart_90_raw
smart_187_normalized
smart_187_raw
smart_188_normalized
smart_188_raw
smart_197_normalized
smart_197_raw
smart_198_normalized
smart_198_raw

In [7]:
```
# ============================================================
# STEP 27 — CATEGORICAL FEATURES
# ============================================================

categorical_result = pd.DataFrame({

    "Categorical_Feature":
        CATEGORICAL_COLUMNS
})

show_and_save(
    categorical_result,
    "step_27_categorical_features.csv",
    "STEP 27 — CATEGORICAL FEATURES"
)
```

```
======================================================================
STEP 27 — CATEGORICAL FEATURES
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Categorical_Feature</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

date	
serial_number	
model	

```
Saved CSV:
output/eda_results\step_27_categorical_features.csv
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Categorical_Feature</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

date	
serial_number	
model	

In [8]:
```
# ============================================================
# STEP 28 — DESCRIPTIVE STATISTICS
# MEMORY-SAFE
# ============================================================

print("\n" + "=" * 70)
print("STEP 28 — DESCRIPTIVE STATISTICS")
print("=" * 70)

# IMPORTANT:
# We do NOT use df.describe()
#
# We calculate count, mean, std, min and max
# incrementally across chunks.
#
# Quartiles are calculated from a controlled sample.

count_values = pd.Series(
    0.0,
    index=NUMERICAL_COLUMNS
)

sum_values = pd.Series(
    0.0,
    index=NUMERICAL_COLUMNS
)

sum_squares = pd.Series(
    0.0,
    index=NUMERICAL_COLUMNS
)

min_values = pd.Series(
    np.inf,
```

```
    index=NUMERICAL_COLUMNS
)

max_values = pd.Series(
    -np.inf,
    index=NUMERICAL_COLUMNS
)

sample_parts = []

sample_rows = 0

for chunk in pd.read_csv(
    INPUT_FILE,
    chunksize=CHUNK_SIZE,
    usecols=NUMERICAL_COLUMNS
):

    count_values += (
        chunk.count()
    )

    sum_values += (
        chunk.sum()
    )

    sum_squares += (
        (chunk ** 2).sum()
    )

    min_values = pd.concat(
        [
            min_values,
            chunk.min()
        ],
        axis=1
    ).min(axis=1)

    max_values = pd.concat(
        [
            max_values,
            chunk.max()
        ],
        axis=1
    ).max(axis=1)

    # Controlled sample
    if sample_rows < SAMPLE_SIZE:

        remaining = (
            SAMPLE_SIZE
            -
            sample_rows
        )

        take = min(
            remaining,
            len(chunk)
        )
```

```
        if take > 0:

            sample_parts.append(
                chunk.head(take)
            )

            sample_rows += take


mean_values = (
    sum_values
    /
    count_values
)

variance_values = (

    (
        sum_squares
        -
        (
            sum_values ** 2
            /
            count_values
        )
    )
    /
    (count_values - 1)
)

std_values = np.sqrt(
    variance_values
)

sample_numeric = pd.concat(
    sample_parts,
    ignore_index=True
)

quartile_values = (
    sample_numeric
    .quantile(
        [0.25, 0.50, 0.75]
    )
)

descriptive_statistics = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Count":
        count_values.values,

    "Mean":
        mean_values.values,

    "Std_Dev":
        std_values.values,
```

```
    "Minimum":
        min_values.values,

    "25%":
        quartile_values.loc[
            0.25
        ].values,

    "50%":
        quartile_values.loc[
            0.50
        ].values,

    "75%":
        quartile_values.loc[
            0.75
        ].values,

    "Maximum":
        max_values.values
})

show_and_save(
    descriptive_statistics,
    "step_28_descriptive_statistics.csv",
    "STEP 28 — DESCRIPTIVE STATISTICS"
)
```

```
======================================================================
STEP 28 — DESCRIPTIVE STATISTICS
======================================================================

======================================================================
STEP 28 — DESCRIPTIVE STATISTICS
======================================================================
C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\c
ore\arraylike.py:402: RuntimeWarning: invalid value encountered in sqrt
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Count</th> <th>Mean</th> <th>Std_Dev</th> <th>Minimum</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>Maximum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody> </tbody>

capacity_bytes	30943146.0	1.565088e+13	NaN	2.400574e+11	1.400052e+13
failure	30943146.0	3.406247e-05	5.836207e-03	0.000000e+00	0.000000e+00
smart_5_normalized	30734429.0	1.000020e+02	2.901228e+00	1.000000e+00	1.000000e+02
smart_5_raw	30734429.0	5.394698e+01	1.277224e+03	0.000000e+00	0.000000e+00
smart_9_normalized	30903290.0	6.315041e+01	3.495968e+01	1.000000e+00	3.700000e+01
smart_9_raw	30903290.0	3.114653e+04	1.892657e+04	0.000000e+00	1.362700e+04
smart_90_normalized	4224514.0	1.000000e+02	0.000000e+00	1.000000e+02	1.000000e+02
smart_90_raw	4224514.0	5.125004e+11	1.102893e+12	9.878425e+10	4.939212e+11
smart_187_normalized	10267176.0	9.900501e+01	8.084231e+00	1.000000e+00	1.000000e+02
smart_187_raw	10267176.0	7.273351e+00	5.060861e+02	0.000000e+00	0.000000e+00
smart_188_normalized	10240022.0	9.999845e+01	3.627430e-01	1.000000e+00	1.000000e+02
smart_188_raw	10240022.0	1.300678e+09	2.469536e+10	0.000000e+00	0.000000e+00
smart_197_normalized	30167940.0	1.000483e+02	2.565479e+00	1.000000e+00	1.000000e+02
smart_197_raw	30167940.0	4.715522e+00	7.233369e+02	0.000000e+00	0.000000e+00
smart_198_normalized	30705419.0	1.000050e+02	2.406945e+00	1.000000e+00	1.000000e+02
smart_198_raw	30705419.0	3.430095e+00	7.129597e+02	0.000000e+00	0.000000e+00

```
Saved CSV:
output/eda_results\step_28_descriptive_statistics.csv
```

Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Count</th> <th>Mean</th> <th>Std_Dev</th> <th>Minimum</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>Maximum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	30943146.0	1.565088e+13	NaN	2.400574e+11	1.400052e+	
failure	30943146.0	3.406247e-05	5.836207e-03	0.000000e+00	0.000000e+	
smart_5_normalized	30734429.0	1.000020e+02	2.901228e+00	1.000000e+00	1.000000e+	
smart_5_raw	30734429.0	5.394698e+01	1.277224e+03	0.000000e+00	0.000000e+	
smart_9_normalized	30903290.0	6.315041e+01	3.495968e+01	1.000000e+00	3.700000e+	
smart_9_raw	30903290.0	3.114653e+04	1.892657e+04	0.000000e+00	1.362700e+	
smart_90_normalized	4224514.0	1.000000e+02	0.000000e+00	1.000000e+02	1.000000e+	
smart_90_raw	4224514.0	5.125004e+11	1.102893e+12	9.878425e+10	4.939212e+	
smart_187_normalized	10267176.0	9.900501e+01	8.084231e+00	1.000000e+00	1.000000e+	
smart_187_raw	10267176.0	7.273351e+00	5.060861e+02	0.000000e+00	0.000000e+	
smart_188_normalized	10240022.0	9.999845e+01	3.627430e-01	1.000000e+00	1.000000e+	
smart_188_raw	10240022.0	1.300678e+09	2.469536e+10	0.000000e+00	0.000000e+	
smart_197_normalized	30167940.0	1.000483e+02	2.565479e+00	1.000000e+00	1.000000e+	
smart_197_raw	30167940.0	4.715522e+00	7.233369e+02	0.000000e+00	0.000000e+	
smart_198_normalized	30705419.0	1.000050e+02	2.406945e+00	1.000000e+00	1.000000e+	
smart_198_raw	30705419.0	3.430095e+00	7.129597e+02	0.000000e+00	0.000000e+	

In [9]:
```
# ============================================================
# STEP 29 — MEAN
# ============================================================

mean_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Mean":
        mean_values.values
})

show_and_save(
    mean_result,
    "step_29_mean.csv",
    "STEP 29 — MEAN"
)
```

```
======================================================================
STEP 29 — MEAN
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.565088e+13
failure	3.406247e-05
smart_5_normalized	1.000020e+02
smart_5_raw	5.394698e+01
smart_9_normalized	6.315041e+01
smart_9_raw	3.114653e+04
smart_90_normalized	1.000000e+02
smart_90_raw	5.125004e+11
smart_187_normalized	9.900501e+01
smart_187_raw	7.273351e+00
smart_188_normalized	9.999845e+01
smart_188_raw	1.300678e+09
smart_197_normalized	1.000483e+02
smart_197_raw	4.715522e+00
smart_198_normalized	1.000050e+02
smart_198_raw	3.430095e+00

```
Saved CSV:
output/eda_results\step_29_mean.csv
```

Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.565088e+13
failure	3.406247e-05
smart_5_normalized	1.000020e+02
smart_5_raw	5.394698e+01
smart_9_normalized	6.315041e+01
smart_9_raw	3.114653e+04
smart_90_normalized	1.000000e+02
smart_90_raw	5.125004e+11
smart_187_normalized	9.900501e+01
smart_187_raw	7.273351e+00
smart_188_normalized	9.999845e+01
smart_188_raw	1.300678e+09
smart_197_normalized	1.000483e+02
smart_197_raw	4.715522e+00
smart_198_normalized	1.000050e+02
smart_198_raw	3.430095e+00

In [10]:
```
# ============================================================
# STEP 30 — MEDIAN
# ============================================================

median_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Median":
        quartile_values
        .loc[0.50]
        .values
})

show_and_save(
    median_result,
    "step_30_median.csv",
    "STEP 30 — MEDIAN"
)
```

```
======================================================================
STEP 30 — MEDIAN
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Median</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.600090e+13
failure	0.000000e+00
smart_5_normalized	1.000000e+02
smart_5_raw	0.000000e+00
smart_9_normalized	7.600000e+01
smart_9_raw	2.800200e+04
smart_90_normalized	1.000000e+02
smart_90_raw	5.196910e+11
smart_187_normalized	1.000000e+02
smart_187_raw	0.000000e+00
smart_188_normalized	1.000000e+02
smart_188_raw	0.000000e+00
smart_197_normalized	1.000000e+02
smart_197_raw	0.000000e+00
smart_198_normalized	1.000000e+02
smart_198_raw	0.000000e+00

```
Saved CSV:
output/eda_results\step_30_median.csv
```

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Median</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.600090e+13
failure	0.000000e+00
smart_5_normalized	1.000000e+02
smart_5_raw	0.000000e+00
smart_9_normalized	7.600000e+01
smart_9_raw	2.800200e+04
smart_90_normalized	1.000000e+02
smart_90_raw	5.196910e+11
smart_187_normalized	1.000000e+02
smart_187_raw	0.000000e+00
smart_188_normalized	1.000000e+02
smart_188_raw	0.000000e+00
smart_197_normalized	1.000000e+02
smart_197_raw	0.000000e+00
smart_198_normalized	1.000000e+02
smart_198_raw	0.000000e+00

In [11]:
```
# ============================================================
# STEP 31 — MODE
# ============================================================

mode_results = []

for column in CATEGORICAL_COLUMNS:

    mode_value = None
    mode_count = 0

    for chunk in pd.read_csv(
        INPUT_FILE,
        chunksize=CHUNK_SIZE,
        usecols=[column]
    ):

        counts = (
            chunk[column]
            .value_counts()
        )
```

```
        if len(counts) > 0:

            current_value = (
                counts.index[0]
            )

            current_count = (
                counts.iloc[0]
            )

            if current_count > mode_count:

                mode_value = (
                    current_value
                )

                mode_count = (
                    current_count
                )

    mode_results.append({

        "Feature":
            column,

        "Mode":
            mode_value,

        "Frequency":
            mode_count
    })


# Numeric mode from sample
for column in NUMERICAL_COLUMNS:

    mode_series = (
        sample_numeric[column]
        .mode()
    )

    mode_value = (
        mode_series.iloc[0]
        if len(mode_series) > 0
        else np.nan
    )

    mode_results.append({

        "Feature":
            column,

        "Mode":
            mode_value,

        "Frequency":
            (
                sample_numeric[column]
                .value_counts()
                .iloc[0]
```

```
                if len(
                    sample_numeric[column]
                    .value_counts()
                ) > 0
                else 0
            )
    })


mode_result = pd.DataFrame(
    mode_results
)

show_and_save(
    mode_result,
    "step_31_mode.csv",
    "STEP 31 — MODE"
)
```

```
======================================================================
STEP 31 — MODE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mode</th> <th>Frequency</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	2026-01-01	100000
serial_number	000a43e7dee60010	1
model	WDC WUH722222ALE6L4	41451
capacity_bytes	16000900661248	180022
failure	0	499997
smart_5_normalized	100.0	495700
smart_5_raw	0.0	474836
smart_9_normalized	100.0	74603
smart_9_raw	9953.0	261
smart_90_normalized	100.0	85769
smart_90_raw	536870912000.0	5969
smart_187_normalized	100.0	109711
smart_187_raw	0.0	109700
smart_188_normalized	100.0	114024
smart_188_raw	0.0	106643
smart_197_normalized	100.0	486071
smart_197_raw	0.0	476949
smart_198_normalized	100.0	496560
smart_198_raw	0.0	490943

```
Saved CSV:
output/eda_results\step_31_mode.csv
```

Out[11]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mode</th> <th>Frequency</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	2026-01-01	100000
serial_number	000a43e7dee60010	1
model	WDC WUH722222ALE6L4	41451
capacity_bytes	16000900661248	180022
failure	0	499997
smart_5_normalized	100.0	495700
smart_5_raw	0.0	474836
smart_9_normalized	100.0	74603
smart_9_raw	9953.0	261
smart_90_normalized	100.0	85769
smart_90_raw	536870912000.0	5969
smart_187_normalized	100.0	109711
smart_187_raw	0.0	109700
smart_188_normalized	100.0	114024
smart_188_raw	0.0	106643
smart_197_normalized	100.0	486071
smart_197_raw	0.0	476949
smart_198_normalized	100.0	496560
smart_198_raw	0.0	490943

In [12]:
```
# ============================================================
# STEP 32 — STANDARD DEVIATION
# ============================================================

std_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Standard_Deviation":
        std_values.values
})

show_and_save(
    std_result,
```

```
    "step_32_standard_deviation.csv",
    "STEP 32 — STANDARD DEVIATION"
)
```

```
======================================================================
STEP 32 — STANDARD DEVIATION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Standard_Deviation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	NaN
failure	5.836207e-03
smart_5_normalized	2.901228e+00
smart_5_raw	1.277224e+03
smart_9_normalized	3.495968e+01
smart_9_raw	1.892657e+04
smart_90_normalized	0.000000e+00
smart_90_raw	1.102893e+12
smart_187_normalized	8.084231e+00
smart_187_raw	5.060861e+02
smart_188_normalized	3.627430e-01
smart_188_raw	2.469536e+10
smart_197_normalized	2.565479e+00
smart_197_raw	7.233369e+02
smart_198_normalized	2.406945e+00
smart_198_raw	7.129597e+02

```
Saved CSV:
output/eda_results\step_32_standard_deviation.csv
```

Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Standard_Deviation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	NaN
failure	5.836207e-03
smart_5_normalized	2.901228e+00
smart_5_raw	1.277224e+03
smart_9_normalized	3.495968e+01
smart_9_raw	1.892657e+04
smart_90_normalized	0.000000e+00
smart_90_raw	1.102893e+12
smart_187_normalized	8.084231e+00
smart_187_raw	5.060861e+02
smart_188_normalized	3.627430e-01
smart_188_raw	2.469536e+10
smart_197_normalized	2.565479e+00
smart_197_raw	7.233369e+02
smart_198_normalized	2.406945e+00
smart_198_raw	7.129597e+02

In [13]:
```
# ============================================================
# STEP 33 — VARIANCE
# ============================================================

variance_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Variance":
        variance_values.values
})

show_and_save(
    variance_result,
    "step_33_variance.csv",
    "STEP 33 — VARIANCE"
)
```

```
======================================================================
STEP 33 — VARIANCE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Variance</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	-2.449499e+26
failure	3.406131e-05
smart_5_normalized	8.417122e+00
smart_5_raw	1.631302e+06
smart_9_normalized	1.222179e+03
smart_9_raw	3.582149e+08
smart_90_normalized	0.000000e+00
smart_90_raw	1.216373e+24
smart_187_normalized	6.535479e+01
smart_187_raw	2.561231e+05
smart_188_normalized	1.315825e-01
smart_188_raw	6.098608e+20
smart_197_normalized	6.581685e+00
smart_197_raw	5.232163e+05
smart_198_normalized	5.793385e+00
smart_198_raw	5.083115e+05

```
Saved CSV:
output/eda_results\step_33_variance.csv
```

Out[13]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Variance</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	-2.449499e+26
failure	3.406131e-05
smart_5_normalized	8.417122e+00
smart_5_raw	1.631302e+06
smart_9_normalized	1.222179e+03
smart_9_raw	3.582149e+08
smart_90_normalized	0.000000e+00
smart_90_raw	1.216373e+24
smart_187_normalized	6.535479e+01
smart_187_raw	2.561231e+05
smart_188_normalized	1.315825e-01
smart_188_raw	6.098608e+20
smart_197_normalized	6.581685e+00
smart_197_raw	5.232163e+05
smart_198_normalized	5.793385e+00
smart_198_raw	5.083115e+05

In [14]:
```
# ============================================================
# STEP 34 — MINIMUM
# ============================================================

minimum_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Minimum":
        min_values.values
})

show_and_save(
    minimum_result,
    "step_34_minimum.csv",
    "STEP 34 — MINIMUM"
)
```

```
======================================================================
STEP 34 — MINIMUM
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Minimum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.400574e+11
failure	0.000000e+00
smart_5_normalized	1.000000e+00
smart_5_raw	0.000000e+00
smart_9_normalized	1.000000e+00
smart_9_raw	0.000000e+00
smart_90_normalized	1.000000e+02
smart_90_raw	9.878425e+10
smart_187_normalized	1.000000e+00
smart_187_raw	0.000000e+00
smart_188_normalized	1.000000e+00
smart_188_raw	0.000000e+00
smart_197_normalized	1.000000e+00
smart_197_raw	0.000000e+00
smart_198_normalized	1.000000e+00
smart_198_raw	0.000000e+00

```
Saved CSV:
output/eda_results\step_34_minimum.csv
```

Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Minimum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.400574e+11
failure	0.000000e+00
smart_5_normalized	1.000000e+00
smart_5_raw	0.000000e+00
smart_9_normalized	1.000000e+00
smart_9_raw	0.000000e+00
smart_90_normalized	1.000000e+02
smart_90_raw	9.878425e+10
smart_187_normalized	1.000000e+00
smart_187_raw	0.000000e+00
smart_188_normalized	1.000000e+00
smart_188_raw	0.000000e+00
smart_197_normalized	1.000000e+00
smart_197_raw	0.000000e+00
smart_198_normalized	1.000000e+00
smart_198_raw	0.000000e+00

In [15]:
```
# ============================================================
# STEP 35 — MAXIMUM
# ============================================================

maximum_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Maximum":
        max_values.values
})

show_and_save(
    maximum_result,
    "step_35_maximum.csv",
    "STEP 35 — MAXIMUM"
)
```

```
======================================================================
STEP 35 — MAXIMUM
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Maximum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.600066e+13
failure	1.000000e+00
smart_5_normalized	2.520000e+02
smart_5_raw	6.552800e+04
smart_9_normalized	1.000000e+02
smart_9_raw	1.132460e+05
smart_90_normalized	1.000000e+02
smart_90_raw	2.814707e+14
smart_187_normalized	1.000000e+02
smart_187_raw	6.553500e+04
smart_188_normalized	1.000000e+02
smart_188_raw	2.860492e+12
smart_197_normalized	2.520000e+02
smart_197_raw	4.617600e+05
smart_198_normalized	2.520000e+02
smart_198_raw	4.617600e+05

```
Saved CSV:
output/eda_results\step_35_maximum.csv
```

Out[15]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Maximum</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.600066e+13
failure	1.000000e+00
smart_5_normalized	2.520000e+02
smart_5_raw	6.552800e+04
smart_9_normalized	1.000000e+02
smart_9_raw	1.132460e+05
smart_90_normalized	1.000000e+02
smart_90_raw	2.814707e+14
smart_187_normalized	1.000000e+02
smart_187_raw	6.553500e+04
smart_188_normalized	1.000000e+02
smart_188_raw	2.860492e+12
smart_197_normalized	2.520000e+02
smart_197_raw	4.617600e+05
smart_198_normalized	2.520000e+02
smart_198_raw	4.617600e+05

In [16]:
```
# ============================================================
# STEP 36 — RANGE
# ============================================================

range_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Range":
        (
            max_values
            -
            min_values
        ).values
})

show_and_save(
    range_result,
    "step_36_range.csv",
    "STEP 36 — RANGE"
)
```

```
======================================================================
STEP 36 — RANGE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Range</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.576060e+13
failure	1.000000e+00
smart_5_normalized	2.510000e+02
smart_5_raw	6.552800e+04
smart_9_normalized	9.900000e+01
smart_9_raw	1.132460e+05
smart_90_normalized	0.000000e+00
smart_90_raw	2.813719e+14
smart_187_normalized	9.900000e+01
smart_187_raw	6.553500e+04
smart_188_normalized	9.900000e+01
smart_188_raw	2.860492e+12
smart_197_normalized	2.510000e+02
smart_197_raw	4.617600e+05
smart_198_normalized	2.510000e+02
smart_198_raw	4.617600e+05

```
Saved CSV:
output/eda_results\step_36_range.csv
```

Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Range</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	2.576060e+13
failure	1.000000e+00
smart_5_normalized	2.510000e+02
smart_5_raw	6.552800e+04
smart_9_normalized	9.900000e+01
smart_9_raw	1.132460e+05
smart_90_normalized	0.000000e+00
smart_90_raw	2.813719e+14
smart_187_normalized	9.900000e+01
smart_187_raw	6.553500e+04
smart_188_normalized	9.900000e+01
smart_188_raw	2.860492e+12
smart_197_normalized	2.510000e+02
smart_197_raw	4.617600e+05
smart_198_normalized	2.510000e+02
smart_198_raw	4.617600e+05

In [17]:
```
# ============================================================
# STEP 37 — QUARTILES
# ============================================================

quartile_result = quartile_values.T.reset_index()

quartile_result.columns = [

    "Feature",

    "25%",

    "50%",

    "75%"
]

show_and_save(
    quartile_result,
    "step_37_quartiles.csv",
    "STEP 37 — QUARTILES"
)
```

```
======================================================================
STEP 37 — QUARTILES
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>25%</th> <th>50%</th> <th>75%</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.400052e+13	1.600090e+13	2.000059e+13
failure	0.000000e+00	0.000000e+00	0.000000e+00
smart_5_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_5_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_9_normalized	3.700000e+01	7.600000e+01	9.800000e+01
smart_9_raw	1.362700e+04	2.800200e+04	4.243275e+04
smart_90_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_90_raw	4.939212e+11	5.196910e+11	5.368709e+11
smart_187_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_187_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_188_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_188_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_197_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_197_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_198_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_198_raw	0.000000e+00	0.000000e+00	0.000000e+00

```
Saved CSV:
output/eda_results\step_37_quartiles.csv
```

Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>25%</th> <th>50%</th> <th>75%</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.400052e+13	1.600090e+13	2.000059e+13
failure	0.000000e+00	0.000000e+00	0.000000e+00
smart_5_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_5_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_9_normalized	3.700000e+01	7.600000e+01	9.800000e+01
smart_9_raw	1.362700e+04	2.800200e+04	4.243275e+04
smart_90_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_90_raw	4.939212e+11	5.196910e+11	5.368709e+11
smart_187_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_187_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_188_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_188_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_197_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_197_raw	0.000000e+00	0.000000e+00	0.000000e+00
smart_198_normalized	1.000000e+02	1.000000e+02	1.000000e+02
smart_198_raw	0.000000e+00	0.000000e+00	0.000000e+00

In [18]:
```
# ============================================================
# STEP 38 — SKEWNESS
# ============================================================

print("\n" + "=" * 70)
print("STEP 38 — SKEWNESS")
print("=" * 70)

# Use controlled sample instead of 30 million rows
skew_values = (
    sample_numeric
    .skew()
)

skew_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Skewness":
        skew_values.values
})
```

```
skew_result = (
    skew_result
    .sort_values(
        "Skewness",
        ascending=False
    )
)

show_and_save(
    skew_result,
    "step_38_skewness.csv",
    "STEP 38 — SKEWNESS"
)
```

```
======================================================================
STEP 38 — SKEWNESS
======================================================================

======================================================================
STEP 38 — SKEWNESS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Skewness</th> </thead> <tbody> <th>15</th> <th>13</th> <th>1</th> <th>9</th> <th>12</th> <th>11</th> <th>3</th> <th>2</th> <th>14</th> <th>5</th> <th>6</th> <th>0</th> <th>4</th> <th>7</th> <th>8</th> <th>10</th> </tbody> <tbody></tbody>

smart_198_raw	591.309643
smart_197_raw	576.297050
failure	408.245841
smart_187_raw	128.951162
smart_197_normalized	56.018449
smart_188_raw	49.149064
smart_5_raw	42.790391
smart_5_normalized	36.911853
smart_198_normalized	24.040222
smart_9_raw	0.390006
smart_90_normalized	0.000000
capacity_bytes	-0.270883
smart_9_normalized	-0.571204
smart_90_raw	-2.135255
smart_187_normalized	-10.792845
smart_188_normalized	-335.033179

```
Saved CSV:
output/eda_results\step_38_skewness.csv
```

Out[18]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Skewness</th> </thead> <tbody> <th>15</th> <th>13</th> <th>1</th> <th>9</th> <th>12</th> <th>11</th> <th>3</th> <th>2</th> <th>14</th> <th>5</th> <th>6</th> <th>0</th> <th>4</th> <th>7</th> <th>8</th> <th>10</th> </tbody> <tbody></tbody>

smart_198_raw	591.309643
smart_197_raw	576.297050
failure	408.245841
smart_187_raw	128.951162
smart_197_normalized	56.018449
smart_188_raw	49.149064
smart_5_raw	42.790391
smart_5_normalized	36.911853
smart_198_normalized	24.040222
smart_9_raw	0.390006
smart_90_normalized	0.000000
capacity_bytes	-0.270883
smart_9_normalized	-0.571204
smart_90_raw	-2.135255
smart_187_normalized	-10.792845
smart_188_normalized	-335.033179

In [19]:

```
# ============================================================
# STEP 39 — KURTOSIS
# ============================================================

kurtosis_values = (
    sample_numeric
    .kurtosis()
)

kurtosis_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Kurtosis":
        kurtosis_values.values
})

kurtosis_result = (
    kurtosis_result
```

```
    .sort_values(
        "Kurtosis",
        ascending=False
    )
)

show_and_save(
    kurtosis_result,
    "step_39_kurtosis.csv",
    "STEP 39 — KURTOSIS"
)
```

```
======================================================================
STEP 39 — KURTOSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Kurtosis</th> </thead> <tbody> <th>15</th> <th>13</th> <th>1</th> <th>10</th> <th>9</th> <th>14</th> <th>12</th> <th>11</th> <th>2</th> <th>3</th> <th>8</th> <th>7</th> <th>0</th> <th>6</th> <th>5</th> <th>4</th> </tbody> <tbody></tbody>

15	smart_198_raw	380961.310127
13	smart_197_raw	365393.647523
1	failure	166663.333313
10	smart_188_normalized	112787.575078
9	smart_187_raw	17390.090012
14	smart_198_normalized	3542.446920
12	smart_197_normalized	3487.681584
11	smart_188_raw	3358.147135
2	smart_5_normalized	2465.849308
3	smart_5_raw	2104.848041
8	smart_187_normalized	123.228629
7	smart_90_raw	7.523209
0	capacity_bytes	0.991521
6	smart_90_normalized	0.000000
5	smart_9_raw	-0.487124
4	smart_9_normalized	-1.192744

```
Saved CSV:
output/eda_results\step_39_kurtosis.csv
```

Out[19]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Kurtosis</th> </thead> <tbody> <th>15</th> <th>13</th> <th>1</th> <th>10</th> <th>9</th> <th>14</th> <th>12</th> <th>11</th> <th>2</th> <th>3</th> <th>8</th> <th>7</th> <th>0</th> <th>6</th> <th>5</th> <th>4</th> </tbody> <tbody></tbody>

smart_198_raw	380961.310127
smart_197_raw	365393.647523
failure	166663.333313
smart_188_normalized	112787.575078
smart_187_raw	17390.090012
smart_198_normalized	3542.446920
smart_197_normalized	3487.681584
smart_188_raw	3358.147135
smart_5_normalized	2465.849308
smart_5_raw	2104.848041
smart_187_normalized	123.228629
smart_90_raw	7.523209
capacity_bytes	0.991521
smart_90_normalized	0.000000
smart_9_raw	-0.487124
smart_9_normalized	-1.192744

In [20]:
```
# ============================================================
# STEP 40 — DISTRIBUTION ANALYSIS
# ============================================================

distribution_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Mean":
        mean_values.values,

    "Median":
        quartile_values
        .loc[0.50]
        .values,

    "Std_Dev":
        std_values.values,

    "Skewness":
        skew_values.values,
```

```
    "Kurtosis":
        kurtosis_values.values
})

show_and_save(
    distribution_result,
    "step_40_distribution_analysis.csv",
    "STEP 40 — DISTRIBUTION ANALYSIS"
)
```

```
======================================================================
STEP 40 — DISTRIBUTION ANALYSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Median</th> <th>Std_Dev</th> <th>Skewness</th> <th>Kurtosis</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	-0.270883	0.9915
failure	3.406247e-05	0.000000e+00	5.836207e-03	408.245841	166663.3333
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	36.911853	2465.8493
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	42.790391	2104.8480
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	-0.571204	-1.1927
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	0.390006	-0.4871
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	0.000000	0.0000
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	-2.135255	7.5232
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	-10.792845	123.2286
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	128.951162	17390.0900
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	-335.033179	112787.5750
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	49.149064	3358.1471
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	56.018449	3487.6815
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	576.297050	365393.6475
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	24.040222	3542.4469
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	591.309643	380961.3101

```
Saved CSV:
output/eda_results\step_40_distribution_analysis.csv
```

Out[20]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Median</th> <th>Std_Dev</th> <th>Skewness</th> <th>Kurtosis</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	-0.270883	0.99
failure	3.406247e-05	0.000000e+00	5.836207e-03	408.245841	166663.33
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	36.911853	2465.84
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	42.790391	2104.84
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	-0.571204	-1.19
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	0.390006	-0.48
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	0.000000	0.00
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	-2.135255	7.52
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	-10.792845	123.22
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	128.951162	17390.09
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	-335.033179	112787.57
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	49.149064	3358.14
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	56.018449	3487.68
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	576.297050	365393.64
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	24.040222	3542.44
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	591.309643	380961.31

In [21]:
```
# ============================================================
# STEP 41 — DISTRIBUTION INTERPRETATION
# ============================================================

distribution_interpretation = []

for column in NUMERICAL_COLUMNS:

    skew_value = (
        skew_values[column]
    )

    if skew_value > 1:

        interpretation = (
            "Highly positively skewed"
        )

    elif skew_value > 0.5:
```

```
        interpretation = (
            "Moderately positively skewed"
        )

    elif skew_value < -1:

        interpretation = (
            "Highly negatively skewed"
        )

    elif skew_value < -0.5:

        interpretation = (
            "Moderately negatively skewed"
        )

    else:

        interpretation = (
            "Approximately symmetric"
        )

    distribution_interpretation.append({

        "Feature":
            column,

        "Skewness":
            skew_value,

        "Interpretation":
            interpretation
    })


distribution_interpretation_df = pd.DataFrame(
    distribution_interpretation
)

show_and_save(
    distribution_interpretation_df,
    "step_41_distribution_interpretation.csv",
    "STEP 41 — DISTRIBUTION INTERPRETATION"
)
```

```
======================================================================
STEP 41 — DISTRIBUTION INTERPRETATION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Skewness</th> <th>Interpretation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	-0.270883	Approximately symmetric
failure	408.245841	Highly positively skewed
smart_5_normalized	36.911853	Highly positively skewed
smart_5_raw	42.790391	Highly positively skewed
smart_9_normalized	-0.571204	Moderately negatively skewed
smart_9_raw	0.390006	Approximately symmetric
smart_90_normalized	0.000000	Approximately symmetric
smart_90_raw	-2.135255	Highly negatively skewed
smart_187_normalized	-10.792845	Highly negatively skewed
smart_187_raw	128.951162	Highly positively skewed
smart_188_normalized	-335.033179	Highly negatively skewed
smart_188_raw	49.149064	Highly positively skewed
smart_197_normalized	56.018449	Highly positively skewed
smart_197_raw	576.297050	Highly positively skewed
smart_198_normalized	24.040222	Highly positively skewed
smart_198_raw	591.309643	Highly positively skewed

```
Saved CSV:
output/eda_results\step_41_distribution_interpretation.csv
```

Out[21]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Skewness</th> <th>Interpretation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	-0.270883	Approximately symmetric
failure	408.245841	Highly positively skewed
smart_5_normalized	36.911853	Highly positively skewed
smart_5_raw	42.790391	Highly positively skewed
smart_9_normalized	-0.571204	Moderately negatively skewed
smart_9_raw	0.390006	Approximately symmetric
smart_90_normalized	0.000000	Approximately symmetric
smart_90_raw	-2.135255	Highly negatively skewed
smart_187_normalized	-10.792845	Highly negatively skewed
smart_187_raw	128.951162	Highly positively skewed
smart_188_normalized	-335.033179	Highly negatively skewed
smart_188_raw	49.149064	Highly positively skewed
smart_197_normalized	56.018449	Highly positively skewed
smart_197_raw	576.297050	Highly positively skewed
smart_198_normalized	24.040222	Highly positively skewed
smart_198_raw	591.309643	Highly positively skewed

In [22]:
```
# ============================================================
# STEP 42 — UNIQUE VALUE ANALYSIS
# ============================================================

unique_counts = pd.Series(
    0,
    index=DATA_COLUMNS,
    dtype="int64"
)

for chunk in pd.read_csv(
    INPUT_FILE,
    chunksize=CHUNK_SIZE
):

    unique_counts = (
        unique_counts
        .combine(
            chunk.nunique(),
            max
        )
    )
```

```
# For exact unique counts, use sample-based indicator
# and categorical frequency analysis below.

unique_result = pd.DataFrame({

    "Feature":
        DATA_COLUMNS,

    "Observed_Unique_Count_Per_Chunk":
        unique_counts.values
})

show_and_save(
    unique_result,
    "step_42_unique_values.csv",
    "STEP 42 — UNIQUE VALUE ANALYSIS"
)
```

```
======================================================================
STEP 42 — UNIQUE VALUE ANALYSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Observed_Unique_Count_Per_Chunk</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	2
serial_number	100000
model	62
capacity_bytes	19
failure	2
smart_5_normalized	88
smart_5_raw	1369
smart_9_normalized	100
smart_9_raw	14585
smart_90_normalized	1
smart_90_raw	248
smart_187_normalized	100
smart_187_raw	460
smart_188_normalized	8
smart_188_raw	1463
smart_197_normalized	23
smart_197_raw	395
smart_198_normalized	23
smart_198_raw	459

```
Saved CSV:
output/eda_results\step_42_unique_values.csv
```

Out[22]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Observed_Unique_Count_Per_Chunk</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> </tbody> <tbody></tbody>

date	2
serial_number	100000
model	62
capacity_bytes	19
failure	2
smart_5_normalized	88
smart_5_raw	1369
smart_9_normalized	100
smart_9_raw	14585
smart_90_normalized	1
smart_90_raw	248
smart_187_normalized	100
smart_187_raw	460
smart_188_normalized	8
smart_188_raw	1463
smart_197_normalized	23
smart_197_raw	395
smart_198_normalized	23
smart_198_raw	459

In [23]:
```
# ============================================================
# STEP 43 — CATEGORICAL FREQUENCY
# ============================================================

print("\n" + "=" * 70)
print("STEP 43 — CATEGORICAL FREQUENCY ANALYSIS")
print("=" * 70)

categorical_results = []

for column in CATEGORICAL_COLUMNS:

    global_counts = {}

    for chunk in pd.read_csv(
```

```
        INPUT_FILE,
        chunksize=CHUNK_SIZE,
        usecols=[column]
    ):

        counts = (
            chunk[column]
            .value_counts()
        )

        for category, count in counts.items():

            category_key = str(
                category
            )

            global_counts[
                category_key
            ] = (
                global_counts.get(
                    category_key,
                    0
                )
                +
                int(count)
            )

    for category, count in (
        sorted(
            global_counts.items(),
            key=lambda x: x[1],
            reverse=True
        )
    ):

        categorical_results.append({

            "Column":
                column,

            "Category":
                category,

            "Frequency":
                count
        })


categorical_frequency_df = pd.DataFrame(
    categorical_results
)

if len(
    categorical_frequency_df
) > 0:

    show_and_save(
        categorical_frequency_df,
        "step_43_categorical_frequency.csv",
        "STEP 43 — CATEGORICAL FREQUENCY"
```

```
    )

else:

    print(
        "No categorical columns found."
    )
```

```
======================================================================
STEP 43 — CATEGORICAL FREQUENCY ANALYSIS
======================================================================

======================================================================
STEP 43 — CATEGORICAL FREQUENCY
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Column</th> <th>Category</th> <th>Frequency</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>351260</th> <th>351261</th> <th>351262</th> <th>351263</th> <th>351264</th> </tbody> <tbody></tbody>

	date	2026-03-31	691324
	date	2026-03-30	345604
	date	2026-03-26	344751
	date	2026-03-20	344636
	date	2026-03-21	344565
	...	...	...
	model	SSDSCKKB480G8R	91
	model	Samsung SSD 850 PRO 1TB	91
	model	ST1000LM024 HN	91
	model	Samsung SSD 870 EVO 2TB	91
	model	Micron 5400 MTFDDAK480TGA	62

351265 rows × 3 columns

```
Saved CSV:
output/eda_results\step_43_categorical_frequency.csv
```

In [24]:

```
# ============================================================
# STEP 44 — TARGET / FAILURE ANALYSIS
# ============================================================

possible_targets = [

    "failure",

    "failed",

    "disk_failure",

    "disk_failed",
```

```
    "failure_status",

    "target",

    "label"
]

target_column = None

for column in possible_targets:

    if column in DATA_COLUMNS:

        target_column = column

        break


print("\n" + "=" * 70)
print("STEP 44 — TARGET / FAILURE ANALYSIS")
print("=" * 70)

print(
    "Target Column:",
    target_column
)

if target_column:

    target_counts = {}

    for chunk in pd.read_csv(
        INPUT_FILE,
        chunksize=CHUNK_SIZE,
        usecols=[target_column]
    ):

        counts = (
            chunk[target_column]
            .value_counts()
        )

        for category, count in counts.items():

            category_key = str(
                category
            )

            target_counts[
                category_key
            ] = (
                target_counts.get(
                    category_key,
                    0
                )
                +
                int(count)
            )
```

```
    target_result = pd.DataFrame({

        "Category":
            list(
                target_counts.keys()
            ),

        "Count":
            list(
                target_counts.values()
            )
    })


    target_result[
        "Percentage"
    ] = (

        target_result["Count"]
        /
        target_result["Count"].sum()
        *
        100
    )


    show_and_save(
        target_result,
        "step_44_target_distribution.csv",
        "STEP 44 — TARGET DISTRIBUTION"
    )

else:

    print(
        "Target column not detected."
    )
```

```
======================================================================
STEP 44 — TARGET / FAILURE ANALYSIS
======================================================================
Target Column: failure

======================================================================
STEP 44 — TARGET DISTRIBUTION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Category</th> <th>Count</th> <th>Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

0	30942092	99.996594	
1	1054	0.003406	

```
Saved CSV:
output/eda_results\step_44_target_distribution.csv
```

In [25]:
```
# ============================================================
# STEP 45 — CORRELATION MATRIX
# SAMPLE-BASED
# ============================================================

print("\n" + "=" * 70)
print("STEP 45 — CORRELATION MATRIX")
print("=" * 70)

correlation_matrix = (
    sample_numeric
    .corr()
)

correlation_result = (
    correlation_matrix
    .reset_index()
    .rename(
        columns={
            "index": "Feature"
        }
    )
)

show_and_save(
    correlation_result,
    "step_45_correlation_matrix.csv",
    "STEP 45 — CORRELATION MATRIX"
)
```

```
======================================================================
STEP 45 — CORRELATION MATRIX
======================================================================

======================================================================
STEP 45 — CORRELATION MATRIX
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>capacity_bytes</th> <th>failure</th> <th>smart_5_normalized</th> <th>smart_5_raw</th> <th>smart_9_normalized</th> <th>smart_9_raw</th> <th>smart_90_normalized</th> <th>smart_90_raw</th> <th>smart_187_normalized</th> <th>smart_187_raw</th> <th>smart_188_normalized</th> <th>smart_188_raw</th> <th>smart_197_normalized</th> <th>smart_197_raw</th> <th>smart_198_normalized</th> <th>smart_198_raw</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.000000	-1.170339e-03	-0.044209	-0.049517	0.408874	-0.718788
failure	-0.001170	1.000000e+00	-0.000004	0.000638	-0.001211	0.001900
smart_5_normalized	-0.044209	-4.278201e-06	1.000000	-0.260499	0.002670	0.032552
smart_5_raw	-0.049517	6.379738e-04	-0.260499	1.000000	-0.020218	0.042585
smart_9_normalized	0.408874	-1.211459e-03	0.002670	-0.020218	1.000000	-0.603838
smart_9_raw	-0.718788	1.900247e-03	0.032552	0.042585	-0.603838	1.000000
smart_90_normalized	NaN	NaN	NaN	NaN	NaN	NaN
smart_90_raw	-0.204363	NaN	-0.000805	-0.001871	-0.017267	0.018553
smart_187_normalized	0.061996	-3.669892e-02	0.379802	-0.503034	0.066647	-0.065720
smart_187_raw	-0.000435	5.435169e-03	-0.095371	0.075196	-0.002043	0.001812
smart_188_normalized	-0.001927	9.805642e-06	0.149877	-0.005265	-0.004840	0.004906
smart_188_raw	-0.007441	1.177507e-01	-0.009070	0.007347	-0.017382	0.017248
smart_197_normalized	-0.066184	-4.109214e-05	0.876776	-0.002753	-0.006303	0.050801
smart_197_raw	-0.002216	-1.490544e-05	-0.054507	0.009653	0.002660	0.000466
smart_198_normalized	-0.034509	5.176299e-07	0.552612	-0.004290	0.016672	0.022870
smart_198_raw	-0.001968	-1.030102e-05	-0.053962	0.009824	0.001086	0.000303

```
Saved CSV:
output/eda_results\step_45_correlation_matrix.csv
```

Out[25]:

capacity_bytes	1.000000	-1.170339e-03	-0.044209	-0.049517	0.408874	-0.718788
failure	-0.001170	1.000000e+00	-0.000004	0.000638	-0.001211	0.001900
smart_5_normalized	-0.044209	-4.278201e-06	1.000000	-0.260499	0.002670	0.032552
smart_5_raw	-0.049517	6.379738e-04	-0.260499	1.000000	-0.020218	0.042585
smart_9_normalized	0.408874	-1.211459e-03	0.002670	-0.020218	1.000000	-0.603838
smart_9_raw	-0.718788	1.900247e-03	0.032552	0.042585	-0.603838	1.000000
smart_90_normalized	NaN	NaN	NaN	NaN	NaN	NaN
smart_90_raw	-0.204363	NaN	-0.000805	-0.001871	-0.017267	0.018553
smart_187_normalized	0.061996	-3.669892e-02	0.379802	-0.503034	0.066647	-0.065720
smart_187_raw	-0.000435	5.435169e-03	-0.095371	0.075196	-0.002043	0.001812
smart_188_normalized	-0.001927	9.805642e-06	0.149877	-0.005265	-0.004840	0.004906
smart_188_raw	-0.007441	1.177507e-01	-0.009070	0.007347	-0.017382	0.017248
smart_197_normalized	-0.066184	-4.109214e-05	0.876776	-0.002753	-0.006303	0.050801
smart_197_raw	-0.002216	-1.490544e-05	-0.054507	0.009653	0.002660	0.000466
smart_198_normalized	-0.034509	5.176299e-07	0.552612	-0.004290	0.016672	0.022870
smart_198_raw	-0.001968	-1.030102e-05	-0.053962	0.009824	0.001086	0.000303

In [26]:
```
# ============================================================
# STEP 46 — STRONG CORRELATIONS
# ============================================================
```

```
correlation_pairs = []

for i in range(
    len(
        correlation_matrix.columns
    )
):

    for j in range(
        i + 1,
        len(
            correlation_matrix.columns
        )
    ):

        feature_1 = (
            correlation_matrix
            .columns[i]
        )

        feature_2 = (
            correlation_matrix
            .columns[j]
        )

        correlation_value = (
            correlation_matrix
            .iloc[
                i,
                j
            ]
        )

        correlation_pairs.append({

            "Feature_1":
                feature_1,

            "Feature_2":
                feature_2,

            "Correlation":
                correlation_value,

            "Absolute_Correlation":
                abs(
                    correlation_value
                )
        })


correlation_pairs_df = pd.DataFrame(
    correlation_pairs
)

correlation_pairs_df = (
    correlation_pairs_df
    .sort_values(
        "Absolute_Correlation",
```

```
        ascending=False
    )
)

show_and_save(
    correlation_pairs_df,
    "step_46_strong_correlations.csv",
    "STEP 46 — STRONG CORRELATIONS"
)
```

```
======================================================================
STEP 46 — STRONG CORRELATIONS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature_1</th> <th>Feature_2</th> <th>Correlation</th> <th>Absolute_Correlation</th> </thead> <tbody> <th>118</th> <th>107</th> <th>109</th> <th>38</th> <th>4</th> <th>...</th> <th>85</th> <th>86</th> <th>87</th> <th>88</th> <th>90</th> </tbody> <tbody></tbody>

smart_197_raw	smart_198_raw	0.994385	0.994385
smart_188_normalized	smart_197_raw	-0.932336	0.932336
smart_188_normalized	smart_198_raw	-0.932336	0.932336
smart_5_normalized	smart_197_normalized	0.876776	0.876776
capacity_bytes	smart_9_raw	-0.718788	0.718788
...	...	...	...
smart_90_raw	smart_187_raw	NaN	NaN
smart_90_raw	smart_188_normalized	NaN	NaN
smart_90_raw	smart_188_raw	NaN	NaN
smart_90_raw	smart_197_normalized	NaN	NaN
smart_90_raw	smart_198_normalized	NaN	NaN

120 rows × 4 columns

```
Saved CSV:
output/eda_results\step_46_strong_correlations.csv
```

Out[26]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature_1</th> <th>Feature_2</th> <th>Correlation</th> <th>Absolute_Correlation</th> </thead> <tbody> <th>118</th> <th>107</th> <th>109</th> <th>38</th> <th>4</th> <th>...</th> <th>85</th> <th>86</th> <th>87</th> <th>88</th> <th>90</th> </tbody> <tbody></tbody>

smart_197_raw	smart_198_raw	0.994385	0.994385
smart_188_normalized	smart_197_raw	-0.932336	0.932336
smart_188_normalized	smart_198_raw	-0.932336	0.932336
smart_5_normalized	smart_197_normalized	0.876776	0.876776
capacity_bytes	smart_9_raw	-0.718788	0.718788
...	...	...	...
smart_90_raw	smart_187_raw	NaN	NaN
smart_90_raw	smart_188_normalized	NaN	NaN
smart_90_raw	smart_188_raw	NaN	NaN
smart_90_raw	smart_197_normalized	NaN	NaN
smart_90_raw	smart_198_normalized	NaN	NaN

120 rows × 4 columns

In [27]:
```
# ============================================================
# STEP 47 — FAILURE CORRELATIONS
# ============================================================

print("\n" + "=" * 70)
print("STEP 47 — FEATURES CORRELATED WITH FAILURE")
print("=" * 70)

if target_column:

    # Convert target to numeric if possible
    target_numeric = pd.to_numeric(
        sample_df[target_column],
        errors="coerce"
    )

    if target_column in sample_numeric.columns:

        failure_correlations = (
            sample_numeric
            .corr()[
                target_column
            ]
            .drop(
                target_column
            )
            .sort_values(
                key=abs,
                ascending=False
```

```
            )
        )

        failure_result = pd.DataFrame({

            "Feature":
                failure_correlations
                .index,

            "Correlation":
                failure_correlations
                .values
        })

        show_and_save(
            failure_result,
            "step_47_failure_correlations.csv",
            "STEP 47 — FAILURE CORRELATIONS"
        )

    else:

        print(
            "Target column is not numeric."
        )

else:

    print(
        "Target column not detected."
    )
```

```
======================================================================
STEP 47 — FEATURES CORRELATED WITH FAILURE
======================================================================

======================================================================
STEP 47 — FAILURE CORRELATIONS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Correlation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> </tbody> <tbody></tbody>

smart_188_raw	1.177507e-01
smart_187_normalized	-3.669892e-02
smart_187_raw	5.435169e-03
smart_9_raw	1.900247e-03
smart_9_normalized	-1.211459e-03
capacity_bytes	-1.170339e-03
smart_5_raw	6.379738e-04
smart_197_normalized	-4.109214e-05
smart_197_raw	-1.490544e-05
smart_198_raw	-1.030102e-05
smart_188_normalized	9.805642e-06
smart_5_normalized	-4.278201e-06
smart_198_normalized	5.176299e-07
smart_90_normalized	NaN
smart_90_raw	NaN

```
Saved CSV:
output/eda_results\step_47_failure_correlations.csv
```

In [28]:
```
# ============================================================
# STEP 48 — ANOMALY DETECTION
# SAMPLE-BASED Z-SCORE
# ============================================================

anomaly_results = []

sample_mean = (
    sample_numeric
    .mean()
)

sample_std = (
    sample_numeric
    .std()
)

for column in NUMERICAL_COLUMNS:

    if sample_std[column] != 0:

        z_scores = (
```

```
            (
                sample_numeric[column]
                -
                sample_mean[column]
            )
            /
            sample_std[column]
        )

        anomaly_count = (
            abs(z_scores) > 3
        ).sum()

    else:

        anomaly_count = 0


    anomaly_results.append({

        "Feature":
            column,

        "Sample_Anomalies_ZScore_GT_3":
            int(
                anomaly_count
            )
    })


anomaly_result = pd.DataFrame(
    anomaly_results
)

show_and_save(
    anomaly_result,
    "step_48_anomaly_analysis.csv",
    "STEP 48 — ANOMALY DETECTION"
)
```

```
======================================================================
STEP 48 — ANOMALY DETECTION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Sample_Anomalies_ZScore_GT_3</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	5510
failure	3
smart_5_normalized	699
smart_5_raw	862
smart_9_normalized	0
smart_9_raw	133
smart_90_normalized	0
smart_90_raw	1603
smart_187_normalized	1174
smart_187_raw	27
smart_188_normalized	7
smart_188_raw	384
smart_197_normalized	179
smart_197_raw	98
smart_198_normalized	180
smart_198_raw	42

```
Saved CSV:
output/eda_results\step_48_anomaly_analysis.csv
```

Out[28]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Sample_Anomalies_ZScore_GT_3</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	5510
failure	3
smart_5_normalized	699
smart_5_raw	862
smart_9_normalized	0
smart_9_raw	133
smart_90_normalized	0
smart_90_raw	1603
smart_187_normalized	1174
smart_187_raw	27
smart_188_normalized	7
smart_188_raw	384
smart_197_normalized	179
smart_197_raw	98
smart_198_normalized	180
smart_198_raw	42

In [29]:
```
# ============================================================
# STEP 49 — FEATURE VARIABILITY
# ============================================================

variability_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Mean":
        mean_values.values,

    "Std_Dev":
        std_values.values,

    "Variance":
        variance_values.values
})

variability_result[
    "Coefficient_of_Variation"
] = (
```

```
    variability_result[
        "Std_Dev"
    ]

    /

    variability_result[
        "Mean"
    ].replace(
        0,
        np.nan
    )
)

variability_result = (
    variability_result
    .sort_values(
        "Coefficient_of_Variation",
        ascending=False
    )
)

show_and_save(
    variability_result,
    "step_49_feature_variability.csv",
    "STEP 49 — FEATURE VARIABILITY"
)
```

```
======================================================================
STEP 49 — FEATURE VARIABILITY
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Std_Dev</th> <th>Variance</th> <th>Coefficient_of_Variation</th> </thead> <tbody> <th>15</th> <th>1</th> <th>13</th> <th>9</th> <th>3</th> <th>11</th> <th>7</th> <th>5</th> <th>4</th> <th>8</th> <th>2</th> <th>12</th> <th>14</th> <th>10</th> <th>6</th> <th>0</th> </tbody> <tbody></tbody>

smart_198_raw	3.430095e+00	7.129597e+02	5.083115e+05	207.854177
failure	3.406247e-05	5.836207e-03	3.406131e-05	171.338333
smart_197_raw	4.715522e+00	7.233369e+02	5.232163e+05	153.394859
smart_187_raw	7.273351e+00	5.060861e+02	2.561231e+05	69.580872
smart_5_raw	5.394698e+01	1.277224e+03	1.631302e+06	23.675548
smart_188_raw	1.300678e+09	2.469536e+10	6.098608e+20	18.986536
smart_90_raw	5.125004e+11	1.102893e+12	1.216373e+24	2.151985
smart_9_raw	3.114653e+04	1.892657e+04	3.582149e+08	0.607662
smart_9_normalized	6.315041e+01	3.495968e+01	1.222179e+03	0.553594
smart_187_normalized	9.900501e+01	8.084231e+00	6.535479e+01	0.081655
smart_5_normalized	1.000020e+02	2.901228e+00	8.417122e+00	0.029012
smart_197_normalized	1.000483e+02	2.565479e+00	6.581685e+00	0.025642
smart_198_normalized	1.000050e+02	2.406945e+00	5.793385e+00	0.024068
smart_188_normalized	9.999845e+01	3.627430e-01	1.315825e-01	0.003627
smart_90_normalized	1.000000e+02	0.000000e+00	0.000000e+00	0.000000
capacity_bytes	1.565088e+13	NaN	-2.449499e+26	NaN

```
Saved CSV:
output/eda_results\step_49_feature_variability.csv
```

Out[29]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Std_Dev</th> <th>Variance</th> <th>Coefficient_of_Variation</th> </thead> <tbody> <th>15</th> <th>1</th> <th>13</th> <th>9</th> <th>3</th> <th>11</th> <th>7</th> <th>5</th> <th>4</th> <th>8</th> <th>2</th> <th>12</th> <th>14</th> <th>10</th> <th>6</th> <th>0</th> </tbody> <tbody></tbody>

smart_198_raw	3.430095e+00	7.129597e+02	5.083115e+05	207.854177
failure	3.406247e-05	5.836207e-03	3.406131e-05	171.338333
smart_197_raw	4.715522e+00	7.233369e+02	5.232163e+05	153.394859
smart_187_raw	7.273351e+00	5.060861e+02	2.561231e+05	69.580872
smart_5_raw	5.394698e+01	1.277224e+03	1.631302e+06	23.675548
smart_188_raw	1.300678e+09	2.469536e+10	6.098608e+20	18.986536
smart_90_raw	5.125004e+11	1.102893e+12	1.216373e+24	2.151985
smart_9_raw	3.114653e+04	1.892657e+04	3.582149e+08	0.607662
smart_9_normalized	6.315041e+01	3.495968e+01	1.222179e+03	0.553594
smart_187_normalized	9.900501e+01	8.084231e+00	6.535479e+01	0.081655
smart_5_normalized	1.000020e+02	2.901228e+00	8.417122e+00	0.029012
smart_197_normalized	1.000483e+02	2.565479e+00	6.581685e+00	0.025642
smart_198_normalized	1.000050e+02	2.406945e+00	5.793385e+00	0.024068
smart_188_normalized	9.999845e+01	3.627430e-01	1.315825e-01	0.003627
smart_90_normalized	1.000000e+02	0.000000e+00	0.000000e+00	0.000000
capacity_bytes	1.565088e+13	NaN	-2.449499e+26	NaN

In [30]:
```
# ============================================================
# STEP 50 — FEATURE MEANS BY FAILURE
# SAMPLE-BASED
# ============================================================

print("\n" + "=" * 70)
print("STEP 50 — FEATURE MEANS BY FAILURE STATUS")
print("=" * 70)

if target_column:

    # Read only target + numeric columns in chunks
    group_parts = []

    required_columns = (
        NUMERICAL_COLUMNS
        +
        [target_column]
    )

    for chunk in pd.read_csv(
```

```
        INPUT_FILE,
        chunksize=CHUNK_SIZE,
        usecols=required_columns
    ):

        # Take a controlled sample
        if len(group_parts) < 5:

            group_parts.append(
                chunk.head(
                    10_000
                )
            )


    grouped_sample = pd.concat(
        group_parts,
        ignore_index=True
    )

    grouped_means = (

        grouped_sample

        .groupby(
            target_column
        )[
            NUMERICAL_COLUMNS
        ]

        .mean()

        .T

        .reset_index()
    )

    grouped_means = (
        grouped_means
        .rename(
            columns={
                "index":
                    "Feature"
            }
        )
    )

    show_and_save(
        grouped_means,
        "step_50_feature_means_by_failure.csv",
        "STEP 50 — FEATURE MEANS BY FAILURE"
    )

else:

    print(
        "Target column not detected."
    )
```

```
======================================================================
STEP 50 — FEATURE MEANS BY FAILURE STATUS
======================================================================

======================================================================
STEP 50 — FEATURE MEANS BY FAILURE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>failure</th> <th>Feature</th> <th>0</th> <th>1</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.544456e+13	1.600090e+13
failure	0.000000e+00	1.000000e+00
smart_5_normalized	1.001686e+02	1.000000e+02
smart_5_raw	3.258363e+00	0.000000e+00
smart_9_normalized	6.099049e+01	9.700000e+01
smart_9_raw	3.085815e+04	2.612800e+04
smart_90_normalized	1.000000e+02	NaN
smart_90_raw	4.684606e+11	NaN
smart_187_normalized	9.992754e+01	NaN
smart_187_raw	3.458297e-01	NaN
smart_188_normalized	1.000000e+02	NaN
smart_188_raw	2.662694e+08	NaN
smart_197_normalized	1.001925e+02	1.000000e+02
smart_197_raw	2.880735e+00	0.000000e+00
smart_198_normalized	9.997217e+01	1.000000e+02
smart_198_raw	3.707485e-01	0.000000e+00

```
Saved CSV:
output/eda_results\step_50_feature_means_by_failure.csv
```

In [31]:
```python
#============================================================
# STEP 51 — FEATURE MEDIANS BY FAILURE
# SAMPLE-BASED
# ============================================================

print("\n" + "=" * 70)
print("STEP 51 — FEATURE MEDIANS BY FAILURE")
print("=" * 70)

if target_column:

    grouped_medians = (
```

```
        grouped_sample

        .groupby(
            target_column
        )[
            NUMERICAL_COLUMNS
        ]

        .median()

        .T

        .reset_index()
    )

    grouped_medians = (
        grouped_medians
        .rename(
            columns={
                "index":
                    "Feature"
            }
        )
    )

    show_and_save(
        grouped_medians,
        "step_51_feature_medians_by_failure.csv",
        "STEP 51 — FEATURE MEDIANS BY FAILURE"
    )

else:

    print(
        "Target column not detected."
    )
```

```
======================================================================
STEP 51 — FEATURE MEDIANS BY FAILURE
======================================================================

======================================================================
STEP 51 — FEATURE MEDIANS BY FAILURE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>failure</th> <th>Feature</th> <th>0</th> <th>1</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.600090e+13	1.600090e+13
failure	0.000000e+00	1.000000e+00
smart_5_normalized	1.000000e+02	1.000000e+02
smart_5_raw	0.000000e+00	0.000000e+00
smart_9_normalized	7.300000e+01	9.700000e+01
smart_9_raw	2.799400e+04	2.612800e+04
smart_90_normalized	1.000000e+02	NaN
smart_90_raw	4.681514e+11	NaN
smart_187_normalized	1.000000e+02	NaN
smart_187_raw	0.000000e+00	NaN
smart_188_normalized	1.000000e+02	NaN
smart_188_raw	0.000000e+00	NaN
smart_197_normalized	1.000000e+02	1.000000e+02
smart_197_raw	0.000000e+00	0.000000e+00
smart_198_normalized	1.000000e+02	1.000000e+02
smart_198_raw	0.000000e+00	0.000000e+00

```
Saved CSV:
output/eda_results\step_51_feature_medians_by_failure.csv
```

In [32]:
```
# ============================================================
# STEP 52 — TREND ANALYSIS
# ============================================================

print("\n" + "=" * 70)
print("STEP 52 — TREND ANALYSIS")
print("=" * 70)

datetime_columns = []

for column in DATA_COLUMNS:

    column_lower = (
        column.lower()
    )

    if any(

        keyword in column_lower
```

```
        for keyword in [
            "date",
            "time",
            "timestamp"
        ]
    ):

        datetime_columns.append(
            column
        )


trend_result = pd.DataFrame({

    "Detected_Date_Time_Column":
        datetime_columns
})

show_and_save(
    trend_result,
    "step_52_trend_analysis.csv",
    "STEP 52 — TREND ANALYSIS"
)
```

```
======================================================================
STEP 52 — TREND ANALYSIS
======================================================================

======================================================================
STEP 52 — TREND ANALYSIS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Detected_Date_Time_Column</th> </thead> <tbody> <th>0</th> </tbody> <tbody></tbody>

date

```
Saved CSV:
output/eda_results\step_52_trend_analysis.csv
```

Out[32]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Detected_Date_Time_Column</th> </thead> <tbody> <th>0</th> </tbody> <tbody></tbody>

date

In [33]:
```
# ============================================================
# STEP 53 — DATA RANGE ANALYSIS
# ============================================================

range_result = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Minimum":
        min_values.values,
```

```
    "Maximum":
        max_values.values,

    "Range":
        (
            max_values
            -
            min_values
        ).values
})

show_and_save(
    range_result,
    "step_53_data_range.csv",
    "STEP 53 — DATA RANGE"
)
```

```
======================================================================
STEP 53 — DATA RANGE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Minimum</th> <th>Maximum</th> <th>Range</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody> </tbody>

capacity_bytes	2.400574e+11	2.600066e+13	2.576060e+13
failure	0.000000e+00	1.000000e+00	1.000000e+00
smart_5_normalized	1.000000e+00	2.520000e+02	2.510000e+02
smart_5_raw	0.000000e+00	6.552800e+04	6.552800e+04
smart_9_normalized	1.000000e+00	1.000000e+02	9.900000e+01
smart_9_raw	0.000000e+00	1.132460e+05	1.132460e+05
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00
smart_90_raw	9.878425e+10	2.814707e+14	2.813719e+14
smart_187_normalized	1.000000e+00	1.000000e+02	9.900000e+01
smart_187_raw	0.000000e+00	6.553500e+04	6.553500e+04
smart_188_normalized	1.000000e+00	1.000000e+02	9.900000e+01
smart_188_raw	0.000000e+00	2.860492e+12	2.860492e+12
smart_197_normalized	1.000000e+00	2.520000e+02	2.510000e+02
smart_197_raw	0.000000e+00	4.617600e+05	4.617600e+05
smart_198_normalized	1.000000e+00	2.520000e+02	2.510000e+02
smart_198_raw	0.000000e+00	4.617600e+05	4.617600e+05

```
Saved CSV:
output/eda_results\step_53_data_range.csv
```

Out[33]:

	Feature	Minimum	Maximum	Range
0	capacity_bytes	2.400574e+11	2.600066e+13	2.576060e+13
1	failure	0.000000e+00	1.000000e+00	1.000000e+00
2	smart_5_normalized	1.000000e+00	2.520000e+02	2.510000e+02
3	smart_5_raw	0.000000e+00	6.552800e+04	6.552800e+04
4	smart_9_normalized	1.000000e+00	1.000000e+02	9.900000e+01
5	smart_9_raw	0.000000e+00	1.132460e+05	1.132460e+05
6	smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00
7	smart_90_raw	9.878425e+10	2.814707e+14	2.813719e+14
8	smart_187_normalized	1.000000e+00	1.000000e+02	9.900000e+01
9	smart_187_raw	0.000000e+00	6.553500e+04	6.553500e+04
10	smart_188_normalized	1.000000e+00	1.000000e+02	9.900000e+01
11	smart_188_raw	0.000000e+00	2.860492e+12	2.860492e+12
12	smart_197_normalized	1.000000e+00	2.520000e+02	2.510000e+02
13	smart_197_raw	0.000000e+00	4.617600e+05	4.617600e+05
14	smart_198_normalized	1.000000e+00	2.520000e+02	2.510000e+02
15	smart_198_raw	0.000000e+00	4.617600e+05	4.617600e+05

In [34]:

```
# ============================================================
# STEP 54 — STATISTICAL FINDINGS
# ============================================================

statistical_findings = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Mean":
        mean_values.values,

    "Median":
        quartile_values
        .loc[0.50]
        .values,

    "Standard_Deviation":
        std_values.values,
```

```
    "Variance":
        variance_values.values,

    "Skewness":
        skew_values.values,

    "Kurtosis":
        kurtosis_values.values
})

show_and_save(
    statistical_findings,
    "step_54_statistical_findings.csv",
    "STEP 54 — STATISTICAL FINDINGS"
)
```

```
======================================================================
STEP 54 — STATISTICAL FINDINGS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Median</th> <th>Standard_Deviation</th> <th>Variance</th> <th>Skewness</th> <th>Kurtosis</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody> </tbody>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	-2.449499e+26	-0.2708
failure	3.406247e-05	0.000000e+00	5.836207e-03	3.406131e-05	408.2458
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	8.417122e+00	36.9118
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	1.631302e+06	42.7903
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	1.222179e+03	-0.5712
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	3.582149e+08	0.3900
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	0.000000e+00	0.0000
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	1.216373e+24	-2.1352
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	6.535479e+01	-10.7928
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	2.561231e+05	128.9511
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	1.315825e-01	-335.0331
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	6.098608e+20	49.1490
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	6.581685e+00	56.0184
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	5.232163e+05	576.2970
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	5.793385e+00	24.0402
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	5.083115e+05	591.3096

```
Saved CSV:
output/eda_results\step_54_statistical_findings.csv
```

Out[34]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	-2.449499e+26	-0.27(
failure	3.406247e-05	0.000000e+00	5.836207e-03	3.406131e-05	408.24
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	8.417122e+00	36.91
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	1.631302e+06	42.79(
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	1.222179e+03	-0.57
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	3.582149e+08	0.39(
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	0.000000e+00	0.00(
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	1.216373e+24	-2.13
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	6.535479e+01	-10.79
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	2.561231e+05	128.95
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	1.315825e-01	-335.03
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	6.098608e+20	49.14
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	6.581685e+00	56.01
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	5.232163e+05	576.29
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	5.793385e+00	24.04(
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	5.083115e+05	591.30

In [35]:

```
# ============================================================
# STEP 55 — COMPLETE EDA REPORT
# ============================================================

eda_report = pd.DataFrame({

    "Feature":
        NUMERICAL_COLUMNS,

    "Mean":
        mean_values.values,

    "Median":
        quartile_values
        .loc[0.50]
        .values,
```

```
    "Std_Dev":
        std_values.values,

    "Minimum":
        min_values.values,

    "Maximum":
        max_values.values,

    "Range":
        (
            max_values
            -
            min_values
        ).values,

    "Skewness":
        skew_values.values,

    "Kurtosis":
        kurtosis_values.values
})

show_and_save(
    eda_report,
    "step_55_complete_eda_report.csv",
    "STEP 55 — COMPLETE EDA REPORT"
)
```

```
======================================================================
STEP 55 — COMPLETE EDA REPORT
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Median</th> <th>Std_Dev</th> <th>Minimum</th> <th>Maximum</th> <th>Range</th> <th>Skewness</th> <th>Kurtosis</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody> </tbody>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	2.400574e+11	2.600066e-
failure	3.406247e-05	0.000000e+00	5.836207e-03	0.000000e+00	1.000000e-
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	1.000000e+00	2.520000e-
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	0.000000e+00	6.552800e-
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	1.000000e+00	1.000000e-
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	0.000000e+00	1.132460e-
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	1.000000e+02	1.000000e-
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	9.878425e+10	2.814707e-
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	1.000000e+00	1.000000e-
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	0.000000e+00	6.553500e-
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	1.000000e+00	1.000000e-
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	0.000000e+00	2.860492e-
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	1.000000e+00	2.520000e-
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	0.000000e+00	4.617600e-
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	1.000000e+00	2.520000e-
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	0.000000e+00	4.617600e-

```
Saved CSV:
output/eda_results\step_55_complete_eda_report.csv
```

Out[35]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Feature</th> <th>Mean</th> <th>Median</th> <th>Std_Dev</th> <th>Minimum</th> <th>Maximum</th> <th>Range</th> <th>Skewness</th> <th>Kurtosis</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> </tbody> <tbody></tbody>

capacity_bytes	1.565088e+13	1.600090e+13	NaN	2.400574e+11	2.600066
failure	3.406247e-05	0.000000e+00	5.836207e-03	0.000000e+00	1.000000
smart_5_normalized	1.000020e+02	1.000000e+02	2.901228e+00	1.000000e+00	2.520000
smart_5_raw	5.394698e+01	0.000000e+00	1.277224e+03	0.000000e+00	6.552800
smart_9_normalized	6.315041e+01	7.600000e+01	3.495968e+01	1.000000e+00	1.000000
smart_9_raw	3.114653e+04	2.800200e+04	1.892657e+04	0.000000e+00	1.132460
smart_90_normalized	1.000000e+02	1.000000e+02	0.000000e+00	1.000000e+02	1.000000
smart_90_raw	5.125004e+11	5.196910e+11	1.102893e+12	9.878425e+10	2.814707
smart_187_normalized	9.900501e+01	1.000000e+02	8.084231e+00	1.000000e+00	1.000000
smart_187_raw	7.273351e+00	0.000000e+00	5.060861e+02	0.000000e+00	6.553500
smart_188_normalized	9.999845e+01	1.000000e+02	3.627430e-01	1.000000e+00	1.000000
smart_188_raw	1.300678e+09	0.000000e+00	2.469536e+10	0.000000e+00	2.860492
smart_197_normalized	1.000483e+02	1.000000e+02	2.565479e+00	1.000000e+00	2.520000
smart_197_raw	4.715522e+00	0.000000e+00	7.233369e+02	0.000000e+00	4.617600
smart_198_normalized	1.000050e+02	1.000000e+02	2.406945e+00	1.000000e+00	2.520000
smart_198_raw	3.430095e+00	0.000000e+00	7.129597e+02	0.000000e+00	4.617600

In [36]:
```
# ============================================================
# STEP 56 — FINAL EDA SUMMARY
# ============================================================

final_summary = pd.DataFrame({

    "Metric": [

        "Total Rows",

        "Total Columns",

        "Numerical Features",

        "Categorical Features",

        "Total Missing Values",

        "Duplicate Rows",
```

```
        "Target Column",
        "Analysis Sample Size"
    ],
    "Value": [
        total_rows,
        len(DATA_COLUMNS),
        len(NUMERICAL_COLUMNS),
        len(CATEGORICAL_COLUMNS),
        int(
            missing_count.sum()
        ),
        duplicate_count,
        target_column,
        len(sample_numeric)
    ]
})

show_and_save(
    final_summary,
    "step_56_final_eda_summary.csv",
    "STEP 56 — FINAL EDA SUMMARY"
)


# ============================================================
# CLEANUP
# ============================================================

del sample_numeric

del sample_parts

del sample_df

gc.collect()


# ============================================================
# FINAL MESSAGE
# ============================================================

print("\n" + "=" * 70)
print(
    "MEMBER 3 — STEPS 22–56 COMPLETED SUCCESSFULLY"
)
print("=" * 70)

print(
    "\nThe complete dataset was processed "
```

```
    "without loading all 30+ million rows "
    "into RAM."
)

print(
    "\nResults available in:"
)

print(
    "1. Jupyter Notebook"
)

print(
    "2. Terminal"
)

print(
    "3. CSV files:"
)

print(
    OUTPUT_FOLDER
)
```

```
======================================================================
STEP 56 — FINAL EDA SUMMARY
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Metric</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> </tbody> <tbody></tbody>

Metric	Value
Total Rows	30943146
Total Columns	19
Numerical Features	16
Categorical Features	3
Total Missing Values	138718464
Duplicate Rows	345662
Target Column	failure
Analysis Sample Size	500000

```
Saved CSV:
output/eda_results\step_56_final_eda_summary.csv

======================================================================
MEMBER 3 — STEPS 22–56 COMPLETED SUCCESSFULLY
======================================================================

The complete dataset was processed without loading all 30+ million rows into RAM.

Results available in:
1. Jupyter Notebook
2. Terminal
3. CSV files:
output/eda_results
```